OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 18

DATABASE SYSTEMS & INFORMATION RETRIEVAL

SUGGESTED TIME: 4 PERIODS

IN THIS CHAPTER

This chapter introduces the design of relational databases: entity-relationship modelling, normalisation to eliminate redundancy, and querying with SQL. Learners finish by building and querying a complete database connected to a simple data-entry form.

- By the end of this chapter I can explain the limits of flat files and the advantages of a Database Management System (DBMS).
- By the end of this chapter I can build an Entity-Relationship (ER) diagram with entities, attributes, relationships and cardinalities.
- By the end of this chapter I can derive a relational schema (tables, primary keys, foreign keys) from an ER diagram.
- By the end of this chapter I can normalise a relation up to Third Normal Form (1NF, 2NF, 3NF) and justify each step.
- By the end of this chapter I can write SQL statements to create tables with constraints (CREATE TABLE, PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE).
- By the end of this chapter I can insert, update and delete data with INSERT, UPDATE and DELETE while preserving integrity.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. From Flat Files to Relational Modelling (ER Diagrams)
2. From ER Diagram to Normalised Relational Schema
3. Querying & Information Retrieval with SQL
4. Hands-on Project: Building & Binding a Complete Database
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can explain the limits of flat files and the advantages of a Database Management System (DBMS).
- By the end of this chapter I can build an Entity-Relationship (ER) diagram with entities, attributes, relationships and cardinalities.
- By the end of this chapter I can derive a relational schema (tables, primary keys, foreign keys) from an ER diagram.
- By the end of this chapter I can normalise a relation up to Third Normal Form (1NF, 2NF, 3NF) and justify each step.
- By the end of this chapter I can write SQL statements to create tables with constraints (CREATE TABLE, PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE).
- By the end of this chapter I can insert, update and delete data with INSERT, UPDATE and DELETE while preserving integrity.
- By the end of this chapter I can retrieve information with SELECT using WHERE, ORDER BY, aggregate functions, GROUP BY, HAVING and JOIN.
- By the end of this chapter I can bind a simple data-entry form to a database table and explain the client–database interaction.
- By the end of this chapter I can analyse a real Cameroonian case study (school, pharmacy, cooperative) and deliver a working mini-database project.

PREREQUISITES

- Basic data organisation: files, records and fields (Lower Sixth, information systems).
- Spreadsheet tables (rows/columns) and the notion of sorting and filtering data.
- Algorithmic thinking: variables, conditions and logical operators (AND, OR, NOT).
- General computer literacy: using a keyboard-driven tool, saving and organising project files.

1 FROM FLAT FILES TO RELATIONAL MODELLING (ER DIAGRAMS)

Every term, the bursar of a large secondary school in Yaoundé struggles to produce the list of students who have not paid their fees, because names, classes and payments are scattered across paper registers and Excel files with duplicated and contradictory entries. Meanwhile, the MINESUP orientation platform must instantly retrieve the results of thousands of GCE candidates without ever mixing up two students with the same name. How can data be organised so that it is stored once, never contradictory, and retrievable in seconds? The answer lies in relational database modelling and the SQL language — the invisible engine behind banks, hospitals, MTN Mobile Money and the GCE Board itself.

1.1 The Limits of Flat Files and Spreadsheets

Before the advent of database management systems, organisations stored data in flat files (text files, CSV) or spreadsheets (Excel, Calc). While simple to create, these approaches quickly become unmanageable as data volume and complexity grow.

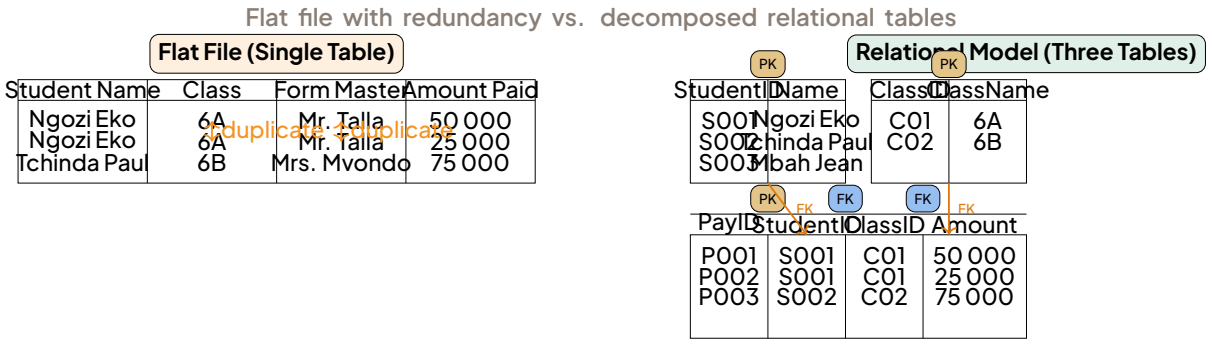
EXPERIMENT **The Bursar's Nightmare**

Imagine the bursar's spreadsheet for fee collection:

Student Name	Class	Form Master	Amount Paid (FCFA)	Date
Ngozi Eko	6A	Mr. Talla	50 000	03/09/2024
Ngozi Eko	6A	Mr. Talla	25 000	15/10/2024
Tchinda Paul	6B	Mrs. Mvondo	75 000	05/09/2024
Tchinda Paul	6B	Mrs. Mvondo	75 000	05/09/2024

Notice the problems:

- Redundancy: Class and Form Master are repeated for every payment row.
- Inconsistency risk: If Mr. Talla is transferred, every row for class 6A must be updated — miss one and the data becomes contradictory.
- Update anomaly: Adding a new student who hasn't paid yet forces a row with empty payment fields (or a dummy entry).
- Deletion anomaly: Deleting the last payment of a student erases their class and form master information.
- No concurrency: Two clerks cannot safely edit the file at the same time.
- Weak security: Anyone with file access sees everything; no row-level permissions.



1.2 Core Concepts: Data, Information, Database, DBMS

DEFINITION **Data, Information, Database, DBMS**

- Data: Raw, unprocessed facts (e.g. `Ngozi, Eko, 50000, 03/09/2024`). Data has no meaning by itself.
- Information: Data that has been processed, organised, or presented in a context that makes it meaningful (e.g. Ngozi Eko paid 50 000 FCFA on 3 September 2024).
- Database: A structured, shared collection of logically related data designed to meet the information needs of an organisation. It represents a mini-world (e.g. a school, a bank).
- DBMS (Database Management System): Software that enables users to define, create, maintain, and control access to a database. It provides data independence, security, concurrency control, backup/recovery, and a query language (usually SQL).

KEY POINTS **Common DBMS Examples**

- MySQL / MariaDB: Widely used open-source relational DBMS; powers many web applications (WordPress, MTN Mobile Money backend services).
- PostgreSQL: Advanced open-source DBMS with strong standards compliance; used in government and GIS systems.
- SQLite: Lightweight, file-based, zero-configuration; embedded in Android phones, browsers, and the GCE Board's offline marking tools.
- Microsoft Access: Desktop DBMS with built-in forms/reports; common in Cameroonian SMEs and school admin offices.
- Oracle Database / Microsoft SQL Server: Enterprise-grade commercial DBMS for

high-volume transaction systems (banks, telecoms).

1.3 The Relational Model

Proposed by Edgar F. Codd in 1970, the relational model represents data as a collection of relations (tables). It is the foundation of virtually all modern DBMS.

DEFINITION **Relational Model Terminology**

- Relation: A two-dimensional table with a unique name. Each relation represents an entity set or a relationship set.
- Tuple (Row): A single row in a relation, representing one instance of an entity or relationship.
- Attribute (Column): A named column in a relation, representing a property of the entity. Each attribute draws values from a domain (a set of permitted values, e.g. `INTEGER`, `VARCHAR(50)`, `DATE`).
- Degree: Number of attributes in a relation.
- Cardinality: Number of tuples in a relation.
- Schema: The logical design of the database: relation names, attribute names, domains, and constraints.
- Instance: The actual data content at a given moment.

PROPERTY **Properties of a Relation**

A relation (table) in the formal relational model obeys strict rules:

1. Atomic values: Each cell contains a single (atomic) value from its domain — no repeating groups, no arrays, no composite values.
2. No duplicate tuples: Every row is unique (guaranteed by a key).
3. Ordering of tuples is insignificant: Rows have no top-to-bottom order.
4. Ordering of attributes is insignificant: Columns can be rearranged without changing the relation.
5. Attribute names are unique within a relation.

1.4 Keys: The Backbone of Relational Integrity

Keys are the mechanism that guarantees uniqueness and creates links between tables.

DEFINITION **Key Types**

- Super Key: A set of one or more attributes that uniquely identifies a tuple. (e.g. {`StudentID`}, {`StudentID, Name`}).
- Candidate Key: A minimal super key — no proper subset is a super key. A relation may have several candidate keys.
- Primary Key (PK): The candidate key chosen by the designer to uniquely identify tuples. Underlined in ER diagrams. Must be `NOT NULL` and unique.
- Composite Key: A primary key consisting of two or more attributes (e.g. {`StudentID`, `SubjectCode`} in a `RESULT` table).
- Foreign Key (FK): An attribute (or set) in one relation that references the primary key of another relation. Enforces referential integrity: every FK value must either match a PK value in the referenced table or be `NULL` (if the relationship is optional).

EXEMPLE **GCE Candidate Number as Primary Key**

The GCE Board assigns each candidate a unique `CandidateNumber` (e.g. `11456/7890`). This is a natural primary key for the `CANDIDATE` entity because:

- It is guaranteed unique by the Board.
- It is stable (never changes during the candidate's lifetime).
- It is familiar to users (printed on timetables, results slips).

In contrast, `Name + DateOfBirth` would be a poor key — two candidates can share both (twins, common names).

KEY POINTS **Primary Key vs Foreign Key — Concrete Example**

Aspect	Primary Key (PK)	Foreign Key (FK)
Purpose	Uniquely identifies a row in its own table	Links a row to a row in another table
Uniqueness	Must be unique	Can have duplicates (many payments → one student)
Null allowed?	No (`NOT NULL`)	Yes, if relationship is optional (e.g. a teacher not yet assigned a class)
Example	`StudentID` in `STUDENT`	`StudentID` in `PAYMENT`
Notation in ER	Underlined attribute	Dashed underline or labelled "FK"

1.5 Entity-Relationship (ER) Modelling

The ER model (Peter Chen, 1976) is a high-level conceptual tool for designing databases before implementing them in a specific DBMS.

DEFINITION **Entities and Attributes**

- Entity: A distinguishable object in the mini-world (e.g. a specific student, a specific class). Represented by a rectangle.
- Entity Set: A collection of entities of the same type (e.g. all students). The rectangle is labelled with the entity set name (singular, capitalised: `STUDENT`).
- Attribute: A property describing an entity. Represented by an oval connected to its entity rectangle.
- Simple vs Composite Attribute: `Name` is simple; `Address` can be composite (`Street`, `City`, `Region`, `PostalCode`). Composite attributes are drawn as an oval with sub-ovals.
- Single-valued vs Multi-valued Attribute: `DateOfBirth` is single-valued; `PhoneNumber` can be multi-valued (a student may have two numbers). Multi-valued attributes are shown with a double oval.
- Derived Attribute: Computed from other attributes (e.g. `Age` from `DateOfBirth`). Shown with a dashed oval.
- Key Attribute: The attribute(s) forming the primary key. Underlined in the oval.

DEFINITION **Relationships and Cardinality**

- Relationship: An association between two or more entity sets. Represented by a diamond connected to the participating entities.
- Cardinality (Mapping Constraints): Specifies how many instances of one entity can be associated with instances of another.
  - One-to-One (1:1): One `STUDENT` has at most one `ID_CARD`; one `ID_CARD` belongs to exactly one `STUDENT`. Notation: `(1,1)` on both sides.
  - One-to-Many (1:N): One `CLASS` contains many `STUDENT`s; each `STUDENT` belongs to exactly one `CLASS`. Notation: `(1,n)` on CLASS side, `(1,1)` on STUDENT side (or `(0,1)` if a student may have no class yet).
  - Many-to-Many (M:N): One `STUDENT` takes many `SUBJECT`s; one `SUBJECT` is taken by many `STUDENT`s. Notation: `(1,n)` on both sides (or `(0,n)` if optional).
- Reading Cardinalities: The pair `(min, max)` on the line next to an entity means: "Each instance of the other entity is linked to at least min and at most max instances of this entity."
  - `(1,1)` = exactly one (mandatory).
  - `(0,1)` = zero or one (optional).
  - `(1,n)` = at least one, up to many (mandatory participation).
  - `(0,n)` = zero to many (optional participation).

METHOD **Steps to Build an ER Diagram**

1. Identify Entity Sets: List the distinct "things" the system must track (nouns in requirements: Student, Class, Teacher, Subject, Payment, Result…).
2. Identify Attributes for Each Entity: For each entity, list its properties. Decide which are simple/composite, single/multi-valued, derived. Choose and underline the primary key.
3. Identify Relationships: Find associations between entities (verbs in requirements: "belongs to", "teaches", "takes", "pays"). Draw diamonds and connect to entities.
4. Determine Cardinalities: For each relationship, ask:
   - "Can an instance of A exist without any B?" → min = 0 or 1.
   - "How many B can one A be linked to?" → max = 1 or n.

   Write `(min,max)` next to each entity on the relationship line.

5. Add Relationship Attributes (if any): Some relationships carry data (e.g. `DateAppointed` on `TEACHES`). Place ovals on the diamond.
6. Review and Refine: Check for redundancy, missing entities, wrong cardinalities. Convert M:N relationships to associative entities (junction tables) later during logical design.

WARNING **Common ER Modelling Traps**

- Confusing an attribute with a relationship: "A student has a class" — `Class` is an entity (it has its own attributes: `ClassName`, `FormMaster`, `Room`), not just an attribute of `STUDENT`. If you model it as an attribute, you lose the ability to list all students in a class without scanning the whole student table.
- Misreading `(0,1)` vs `(1,n)`: `(0,1)` on the `CLASS` side of `BELONGS_TO` means "a class may have zero or one student" (wrong!). The correct cardinality for "a class has many students" is `(1,n)` on the CLASS side and `(1,1)` (or `(0,1)`) on the STUDENT side.
- Forgetting to underline the PK: In GCE exams, missing underlines lose marks. Always underline key attributes in ovals.
- Multi-valued attributes in relational tables: A multi-valued attribute (e.g. `PhoneNumbers`) cannot stay as a single column in a relational table (violates atomicity). It must become a separate entity `PHONE` linked 1:N to `STUDENT`.

DID YOU KNOW? **Exam Tip: Underline Primary Keys**

In every GCE Advanced Level Computer Science paper, ER diagram questions explicitly require: "Underline the primary key attributes." Examiners check this first. Even if the diagram is otherwise perfect, missing underlines can cost 2–3 marks out of 10. Make it a reflex: as soon as you draw an attribute oval that is part of the PK, underline its name.

1.6 Case Study: Modelling a School Database

We now walk through the complete ER modelling process for a simplified school management system covering students, classes, teachers, and subjects.

Requirements (simplified):

- The school has **classes** (e.g. 6A, 6B). Each class has a unique code, a name, a form master (a teacher), and a classroom number.
- **Students** enroll in exactly one class. A student has a unique admission number, name, date of birth, gender, and one or more phone numbers.

- **Teachers** have a unique staff ID, name, qualification, and phone number. A teacher can be form master of at most one class.
- **Subjects** have a unique code and name (e.g. `CS101`, `Computer Science`).
- A **teacher teaches one or more subjects** to one or more classes. A subject in a class is taught by exactly one teacher.
- **Students take subjects** and receive **results** (marks) per term.

Step 1: Entity Sets `STUDENT`, `CLASS`, `TEACHER`, `SUBJECT`, `RESULT` (RESULT is an associative entity for the M:N between STUDENT and SUBJECT, with extra attribute `Mark`).

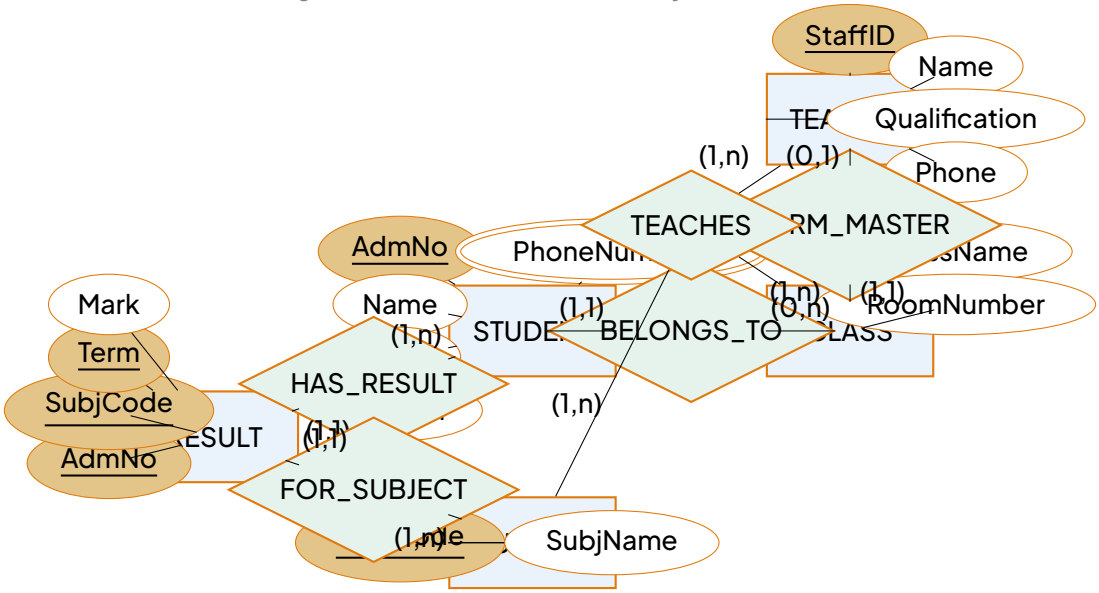
Step 2: Attributes

- `STUDENT`: AdmNo, Name, DOB, Gender. Multi-valued: PhoneNumbers.
- `CLASS`: ClassCode, ClassName, RoomNumber.
- `TEACHER`: StaffID, Name, Qualification, Phone.
- `SUBJECT`: SubjCode, SubjName.
- `RESULT`: AdmNo, SubjCode, Term, Mark. (Composite PK)

Step 3 & 4: Relationships and Cardinalities

- `BELONGS_TO` between `STUDENT` and `CLASS`: `(1,n)` on CLASS side (a class has at least one student? usually yes, but `(0,n)` is safer for a new class), `(1,1)` on STUDENT side (every student belongs to exactly one class).
- `FORM_MASTER` between `TEACHER` and `CLASS`: `(0,1)` on TEACHER (a teacher may not be a form master), `(1,1)` on CLASS (every class has exactly one form master).
- `TEACHES` between `TEACHER`, `SUBJECT`, `CLASS` (ternary relationship) — or better, two binary relationships: `TEACHES_SUBJ` (TEACHER–SUBJECT, M:N) and `ASSIGNED_TO_CLASS` (TEACHER–CLASS, M:N) with a junction entity `TEACHING_ASSIGNMENT` carrying `Year`. For simplicity at GCE level, we often use a ternary diamond with cardinalities `(1,n)` on each entity side.
- `TAKES` between `STUDENT` and `SUBJECT`: M:N → resolved by `RESULT` entity with relationships `HAS_RESULT` (STUDENT–RESULT, 1:N) and `FOR_SUBJECT` (SUBJECT–RESULT, 1:N).

ER Diagram for the School Case Study (Chen notation)



WORKED EXAMPLE 1 **Completing an ER Diagram from a Description**

Statement: A library system tracks `BOOK`, `MEMBER`, and `LOAN`. A book has ISBN (PK), Title, Author, Category. A member has MemberID (PK), Name, Email, Phone (multi-valued). A loan records which member borrowed which book, on which date, and the due date. A member can have many loans; a book can be loaned many times (but only one active loan at a time). A loan must link to exactly one member and one book.
Draw the ER diagram.
Solution:

1. Entities: `BOOK` (ISBN, Title, Author, Category), `MEMBER` (MemberID, Name, Email, `Phone` [double oval]), `LOAN` (LoanID, ISBN, MemberID, LoanDate, DueDate). Note: `LOAN` could use composite PK {`ISBN, MemberID, LoanDate`} but a surrogate `LoanID` is simpler.
2. Relationships:
   - `BORROWS` between `MEMBER` and `LOAN`: `(1,n)` on MEMBER side, `(1,1)` on LOAN side.
   - `LOANS_OUT` between `BOOK` and `LOAN`: `(1,n)` on BOOK side, `(1,1)` on LOAN side.
3. Cardinalities check: A member may have zero loans `(0,n)`? The requirement says "can have many loans" — implies zero is allowed. So `(0,n)` on MEMBER. A book may have zero loans `(0,n)`. A loan must have one member and one book `(1,1)`.
4. Diagram: Draw rectangles for entities, ovals for attributes (underline PKs, double oval for Phone), diamonds for relationships with cardinality labels.

1.7 GCE Exam Focus: Reading and Completing ER Diagrams

In the GCE Advanced Level examination, you will frequently encounter questions that ask you to:

1. **Interpret a given ER diagram:** Write the cardinalities in words, identify primary/foreign keys, list entities and attributes.
2. **Complete a partial ER diagram:** Add missing attributes, relationships, or cardinalities based on a textual description.
3. **Convert an ER diagram to a relational schema:** (Covered in Section 2).

METHOD **Exam Technique: Reading an ER Diagram**

1. Scan entities and keys: Note every rectangle and its underlined attributes. These become tables and PKs.
2. Scan relationships and cardinalities: For each diamond, read the `(min,max)` pairs on both sides. Translate to sentences: "Each CLASS is linked to at least 0 and at most n STUDENTs" → "A class may have zero or many students."
3. Identify FKs: For a 1:N relationship, the FK goes on the "N" side (the entity with `(1,n)` or `(0,n)` on the other side).
4. Spot M:N: If both sides show `(1,n)` or `(0,n)`, a junction table is needed.
5. Check for weak entities: An entity with a dashed rectangle (or double rectangle) and a partial key (dashed underline) depends on an owner entity via an identifying relationship (double diamond). Its PK is composite: owner PK + partial key.

EXERCISE 1 **Practice: School ER Diagram Completion**

The following partial ER diagram describes a simplified `EXAM` system. Complete it by adding:

1. The missing primary key for `CANDIDATE` (use `CandidateNo`).
2. The cardinalities for `SITS_FOR` between `CANDIDATE` and `EXAM_SESSION` (a candidate sits for one session; a session has many candidates).
3. The attributes for `EXAM_SESSION` (`SessionID` PK, `Year`, `Series`).
4. The relationship `GRADED_IN` between `CANDIDATE`, `SUBJECT`, and `EXAM_SESSION` with a `Grade` attribute. Indicate cardinalities.

(Draw your answer on paper. A corrected version appears in the next section's exercises.)

SOLUTION **Exercise 1**

1. `CANDIDATE`: add oval CandidateNo, Name, DOB, CentreNo.
2. `SITS_FOR`: cardinalities `(1,1)` on CANDIDATE side, `(1,n)` on EXAM_SESSION side.
3. `EXAM_SESSION`: ovals SessionID, Year, Series.
4. `GRADED_IN` (ternary diamond): connect to CANDIDATE `(1,1)`, SUBJECT `(1,1)`, EXAM_SESSION `(1,1)`. Add attribute `Grade` on diamond (dashed oval if derived, else normal). In practice, this ternary is often replaced by a `RESULT` entity with composite PK {`CandidateNo`, `SubjCode`, `SessionID`} and attribute `Grade`.

KEY POINTS **Key Takeaways for Section 1**

- Flat files suffer from redundancy, update/delete anomalies, and poor concurrency/security.
- A database is a structured, shared collection of logically related data; a DBMS manages it.
- The relational model uses tables (relations) with rows (tuples) and columns (attributes) over domains.
- Keys ensure integrity: PK uniquely identifies a row; FK links rows across tables via referential integrity.
- ER diagrams model reality at conceptual level: entities (rectangles), attributes (ovals), relationships (diamonds) with `(min,max)` cardinalities.
- Always underline PK attributes. Multi-valued attributes become separate entities in the logical model.
- Reading cardinalities: `(min,max)` on entity E's side of relationship R = "each instance of the other entity participates in at least `min` and at most `max` instances of R with E".

2 FROM ER DIAGRAM TO NORMALISED RELATIONAL SCHEMA

In the previous section we learnt how to model reality with an **Entity–Relationship diagram**: entities such as `STUDENT` or `PAYMENT`, their attributes, and the relationships (one-to-many, many-to-many) that connect them. An ER diagram, however, is only a picture of the data. A real database management system (DBMS) such as MySQL or PostgreSQL can only store **tables**. The work of

this section is therefore twofold:

1. **Translate** the ER diagram into a first set of tables (the **relational schema**);
2. **Refine** those tables so that they do not waste space and do not produce contradictions when data is inserted, modified or deleted. This refinement process is called **normalisation**.

DEFINITION **Relational schema, primary key, foreign key**

The **relational schema** of a database is the list of its tables, each described by its name and its columns, e.g. `STUDENT(stud_id, stud_name, class_id)`. The **primary key** (`PK`) of a table is the column (or minimal set of columns) whose value identifies each row uniquely; it is underlined in the schema. A **foreign key** (`FK`) is a column in one table that refers to the primary key of another table, thereby linking rows across tables.

2.1 Deriving Tables from an ER Diagram

The translation from an ER diagram to tables follows three simple mechanical rules.

METHOD **From ER diagram to relational schema**

1. One table per entity. Each entity type becomes a table; each attribute becomes a column; the identifier of the entity becomes the **primary key** (underlined or marked `PK`).
2. One-to-many relationship ⇒ foreign key. The primary key of the "one" side is copied into the table of the "many" side, where it is called a **foreign key** (`FK`). Example: one `CLASS` has many `STUDENT`s, so `STUDENT` receives the column `class_id` as a foreign key.
3. Many-to-many relationship ⇒ junction table. Create a new table whose primary key is the pair of the two entity keys (plus any attribute of the relationship itself). Example: `STUDENT` and `SUBJECT` are linked by `ENROL(student_id, subject_id, term)`.

WARNING **Many-to-many is not one-to-many**

A very common GCE mistake is to treat a many-to-many relationship as if it were one-to-many. If a student takes many subjects and a subject is taken by many students, putting `subject_id` inside `STUDENT` (or `student_id` inside `SUBJECT`) is wrong: a single row can only hold one value. The only correct solution is a **junction table** with a composite key. Always ask: "can both sides be 'many'?" If yes, create a third table.

2.2 Why Bad Design Hurts: The Three Anomalies

Suppose the bursar of a secondary school in Bamenda keeps school fees receipts in a single spreadsheet-like table:

receipt_no	stud_id	stud_name	class	amount	cashier
R001	S01	Acha B.	Form 5A	25000	Mrs. Ngong
R002	S02	Tabi E.	Form 5A	25000	Mrs. Ngong
R003	S01	Acha B.	Form 5A	10000	Mr. Fuh
R004	S03	Manka'a L.	Lower 6B	30000	Mr. Fuh

This un-normalised design suffers from three classical **anomalies**:

- **Insertion anomaly.** A new student who has not yet paid anything cannot be recorded, because a row requires a `receipt_no`. Similarly, we cannot register a new cashier until she issues a receipt.
- **Update anomaly.** "Acha B." and "Form 5A" are stored on two rows. If Acha's name is corrected on one row only, the database contradicts itself. The more copies of a fact, the more chances to update some and forget others.
- **Deletion anomaly.** If receipt R004 is deleted (e.g. it was cancelled), we lose all trace of student S03 and of the class "Lower 6B", because they appear nowhere else.

KEY POINTS **Anomalies signal bad design**

Redundancy (the same fact stored many times) is the root cause of insertion, update and deletion anomalies. Normalisation removes redundancy by splitting data into well-structured tables linked by foreign keys.

2.3 Functional Dependencies

To decide how to split a table, we need a precise tool: the notion of functional dependency.

DEFINITION **Functional dependency**

Let A and B be attributes (or sets of attributes) of a table. We say that B is **functionally dependent** on A, written

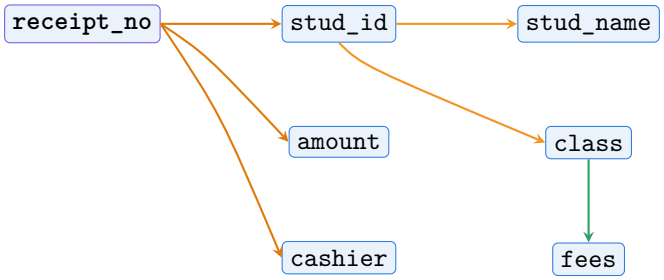
$$A \to B \quad \text{(read "}A\text{ determines }B\text{")},$$

if knowing the value of A always fixes a unique value of B. Formally: whenever two rows agree on A, they must agree on B.

In our receipt table we can read off, from the meaning of the data:

- `receipt_no` → `stud_id, amount, cashier` (a receipt concerns one student, one amount, one cashier);
- `stud_id` → `stud_name, class` (a student has one name and belongs to one class);
- `class` → `fees` (if we add the standard fees per class, e.g. Form 5A → 25000).

Note that `stud_name` → `stud_id` is not valid: two different students may share the same name. Functional dependencies express **business rules**, not coincidences in one small sample of data.



Functional dependency graph for the receipt table. Bold node: key of the whole table; orange arrows: dependencies on a non-key attribute; green arrow: transitive dependency through a non-key attribute.

2.4 The Three Normal Forms

Normalisation is a ladder of tests. Each normal form removes one kind of redundancy.

PROPERTY **1NF, 2NF, 3NF**

- First Normal Form (1NF). Every attribute contains only **atomic** (indivisible) values; there are no repeating groups or lists inside a cell, and each row is unique.
- Second Normal Form (2NF). The table is in 1NF and every non-key attribute depends on the whole primary key, not on part of it (no **partial dependency**). This only concerns tables with a composite key.
- Third Normal Form (3NF). The table is in 2NF and no non-key attribute depends on another non-key attribute (no **transitive dependency**): if $K \to A$ and $A \to B$ with A non-key, then B must be moved out.

(Statements of the normal forms are examinable as definitions; formal proofs are not required at GCE level.)

WARNING **1NF does not imply 2NF — check composite keys first**

A table whose primary key is a single attribute is automatically in 2NF once it is in 1NF: there is no "part" of the key to depend on. Partial dependencies can only exist with a composite key. So when you meet a table keyed by, say, (`stud_id`, `subject_id`), immediately ask whether attributes such as `stud_name` depend only on `stud_id` — if so, the table violates 2NF.

2.5 Worked Example: Normalising the School Fees Receipt Table

We now normalise the bursar's table step by step, adding the column `fees` (the standard fee for the class) to make the transitive dependency visible.

Step 1 — Un-normalised form (UNF). Imagine the bursar first wrote, on one line per student, a list of receipts:

```
S01, Acha B., Form 5A, 25000, {R001: 25000/Mrs. Ngong; R003: 10000/Mr. Fuh}
```

The cell contains a repeating group of receipts: this violates atomicity.

Step 2 — First Normal Form (1NF). Explode the repeating group: one row per receipt, as in the table shown earlier. All values are now atomic. Primary key: `receipt_no`. But redundancy is massive (names, classes and fees repeated).

Step 3 — Second Normal Form (2NF). In our table the key `receipt_no` is a single attribute, so there is no partial dependency and the table is technically already 2NF. To see the 2NF rule in action, consider instead a table ENROL(stud_id, subject_id, stud_name, subject_name, grade): `stud_name` depends only on `stud_id` (part of the key), so we split it into `STUDENT`, `SUBJECT` and ENROL(stud_id, subject_id, grade).

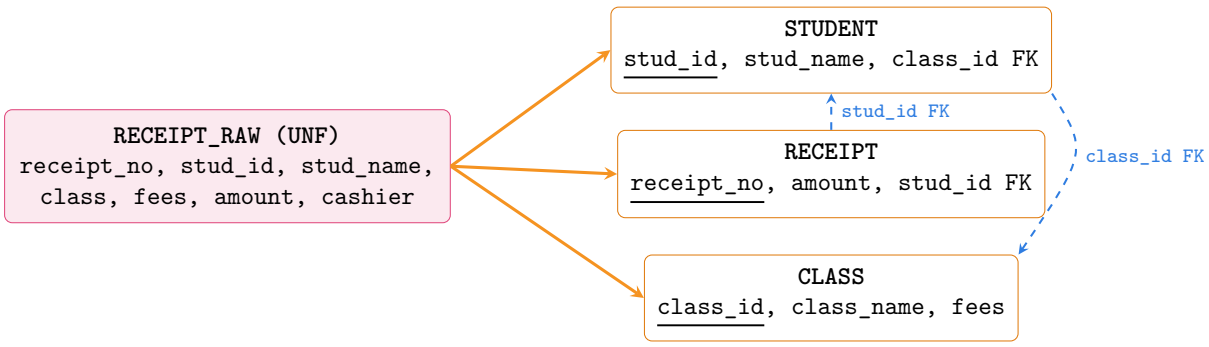
Step 4 — Third Normal Form (3NF). In the receipt table we have the chain

$$\texttt{receipt_no} \rightarrow \texttt{stud_id} \rightarrow \texttt{stud_name, class} \rightarrow \texttt{fees}.$$

`stud_name`, `class` and `fees` depend on `stud_id` and `class`, which are non-key attributes: these are transitive dependencies. We decompose:

- STUDENT(stud_id, stud_name, class_id FK);
- CLASS(class_id, class_name, fees);
- RECEIPT(receipt_no, amount, stud_id FK, cashier_id FK);
- CASHIER(cashier_id, cashier_name).

Each fact is now stored exactly once; the three anomalies have disappeared: a new student can be inserted in `STUDENT` before paying anything; a name is corrected in exactly one row; deleting a receipt no longer erases a student or a class.



Decomposition of the un-normalised receipt table into 3NF tables. Solid orange arrows: decomposition; dashed blue arrows: foreign key references.

METHOD **Normalisation procedure (UNF → 3NF)**

1. UNF → 1NF: remove repeating groups; give every row atomic values; choose a primary key.
2. 1NF → 2NF: if the key is composite, move every attribute that depends on only part of the key into a new table keyed by that part.
3. 2NF → 3NF: find chains $K \to A \to B$ where A is non-key; move A and B into a new table keyed by A, leaving A behind as a foreign key.
4. Check: every non-key attribute must depend on the key, the whole key, and nothing but the key.

WORKED EXAMPLE 2 **GCE-style question: state the normal form and decompose**

Statement. The following table records book loans in a school library:

`LOAN(stud_id, book_id, stud_name, title, loan_date)`

(a) Give two functional dependencies. (b) State the normal form of the table, justifying your answer. (c) Decompose it into 3NF.

SOLUTION

Solution. (a) `stud_id` → `stud_name` and `book_id` → `title`; also (`stud_id`, `book_id`) → `loan_date`.
(b) The table is in 1NF (all values atomic) but not 2NF: `stud_name` depends only on `stud_id` and `title` only on `book_id` — both are partial dependencies on part of the composite key (`stud_id`, `book_id`).
(c) Decomposition into 3NF:

- STUDENT(stud_id, stud_name)
- BOOK(book_id, title)
- LOAN(stud_id, book_id, loan_date) with two foreign keys.

Now every non-key attribute depends on the whole key of its table and nothing else, so all three tables are in 3NF.

EXERCISE 2 **Normalise a canteen orders table**

A school canteen in Douala stores orders as:
ORDER(order_no, date, stud_id, stud_name, class_name, class_fees)

1. List all functional dependencies.
2. Explain why the table is in 2NF but not 3NF.
3. Decompose it into 3NF tables, underlining primary keys and marking foreign keys.

SOLUTION **Exercise: canteen orders**

1. order_no → date, stud_id; stud_id → stud_name, class_name; class_name → class_fees.
2. The key order_no is a single attribute, so no partial dependency is possible: the table is 2NF. But order_no → stud_id → stud_name, class_name and class_name → class_fees are transitive dependencies through non-key attributes, violating 3NF.
3. STUDENT(stud_id, stud_name, class_name FK); CLASS(class_name, class_fees); ORDER(order_no, date, stud_id FK).

KEY POINTS **Key points of the section**

- ER → tables: one table per entity; 1–N via a foreign key; M–N via a junction table.
- Redundancy causes insertion, update and deletion anomalies; normalisation removes it.
- $A \to B$ means: equal A values force equal B values.
- 1NF: atomic values. 2NF: no partial dependency on part of a composite key. 3NF: no transitive dependency through a non-key attribute.

- Golden rule: every non-key attribute depends on the key, the whole key, and nothing but the key.

With our schema now clean and normalised, the next step is to interrogate it: Section 3 shows how SQL turns these tables into answers.

3 QUERYING & INFORMATION RETRIEVAL WITH SQL

In the previous section we transformed an ER diagram into a normalised relational schema: a set of well-structured tables linked by foreign keys. But a schema on paper is only a design. For the database to become useful, we must be able to **create** the tables in a real Database Management System (DBMS), **fill** them with data, and above all **retrieve information** from them. This is exactly the role of **SQL**. Imagine the bursar of a large school in Yaoundé: the database contains thousands of students, classes and payments. The bursar does not want to read every row; he wants to ask questions: "How many students are in Lower Sixth Science?" "Who has not yet paid fees?" SQL is the language in which such questions are asked.

3.1 Introducing SQL: a Declarative Language

DEFINITION **SQL, DDL and DML**

SQL (Structured Query Language) is the standard language used to communicate with a relational DBMS. It is a declarative language: the user states what result is wanted, not how to compute it — the DBMS chooses the execution strategy. SQL commands fall into two main families:

- DDL (Data Definition Language): commands that define the structure of the database — `CREATE`, `ALTER`, `DROP`.
- DML (Data Manipulation Language): commands that work on the data inside the tables — `INSERT`, `UPDATE`, `DELETE`, and the query command `SELECT`.

WARNING **SQL is not case-sensitive, but be consistent**

SQL keywords may be written in upper or lower case (`select` works like `SELECT`). By convention — and for clarity in the GCE examination — we write keywords in CAPITALS and table/column names as defined in the schema. Table and column names, however, must be spelled exactly as created.

Throughout this section we use the school database designed earlier:

Table	Columns (keys underlined)
CLASS	classID, className, level
STUDENT	studentID, fullName, gender, birthDate, feesPaid, classID (FK)

3.2 Creating the Database and Tables (DDL)

Before any data can be stored, the database and its tables must be created. Each column is given a **data type** and optional **constraints**.

```
CREATE DATABASE school;
USE school;
CREATE TABLE CLASS (
  classID INT PRIMARY KEY,
  className VARCHAR(30) NOT NULL,
  level VARCHAR(10) DEFAULT 'Lower Sixth'
);
CREATE TABLE STUDENT (
  studentID INT PRIMARY KEY,
  fullName VARCHAR(60) NOT NULL,
  gender VARCHAR(1),
  birthDate DATE,
  feesPaid DECIMAL(10,2) DEFAULT 0,
  classID INT,
  FOREIGN KEY (classID) REFERENCES CLASS(classID)
);
```

DEFINITION **Common data types and constraints**

Data types: `INT` (whole numbers), `VARCHAR(n)` (text of up to *n* characters), `DATE` (calendar dates, written `'2008-05-17'`), `DECIMAL(m,d)` (exact decimal numbers with *m* digits, *d* after the point — ideal for money in FCFA).
Constraints: `PRIMARY KEY` (uniquely identifies each row, never NULL), `FOREIGN KEY ... REFERENCES` (enforces the link between tables), `NOT NULL` (a value is compulsory), `UNIQUE` (no two rows share the value), `DEFAULT` (value used when none is supplied).

The foreign key constraint is the SQL translation of the relationship in our ER diagram: the DBMS will refuse to insert a student whose `classID` does not exist in `CLASS`. This is **referential integrity**.

3.3 Populating and Modifying Data (DML)

- `INSERT INTO CLASS VALUES (1, 'LS Science A', 'Lower Sixth');`
- `INSERT INTO STUDENT (studentID, fullName, gender, birthDate, classID)`
- `VALUES (101, 'Amina Bello', 'F', '2008-03-12', 1);`
- `UPDATE STUDENT SET feesPaid = 50000 WHERE studentID = 101;`
- `DELETE FROM STUDENT WHERE studentID = 101;`

WARNING **The missing WHERE disaster**

In `UPDATE` and `DELETE`, the `WHERE` clause selects the rows concerned. If you forget it, the command applies to every row of the table: `UPDATE STUDENT SET feesPaid = 0;` wipes the payments of all students, and `DELETE FROM STUDENT;` empties the whole table. Always write and check the `WHERE` clause first.

3.4 Simple Retrieval: SELECT and WHERE

The `SELECT` statement answers questions. Its skeleton is:

- `SELECT column1, column2  -- what you want to see`
- `FROM table  -- where the data lives`
- `WHERE condition;  -- which rows qualify`

`SELECT *` means "all columns". `DISTINCT` removes duplicate values from the result. Conditions use comparison operators (=, <, >, <=, >=, <> for "different") combined with `AND`, `OR`, `NOT`, plus the special operators:

Operator	Meaning	Example
`BETWEEN a AND b`	value in the inclusive range [*a*, *b*]	`feesPaid BETWEEN 20000 AND 50000`
`LIKE`	pattern match; % = any string, _ = one character	`fullName LIKE 'A%'`
`IN (...)`	value belongs to a list	`level IN ('Form 5','LS')`
`IS NULL`	value is missing	`birthDate IS NULL`

WARNING **NULL is not zero**

A missing value (`NULL`) cannot be tested with =. Writing `WHERE birthDate = NULL` returns nothing, even if NULLs exist. Only `IS NULL` / `IS NOT NULL` work.

3.5 Sorting and Limiting: ORDER BY and LIMIT

`ORDER BY` sorts the result on one or more columns, ascending (`ASC`, the default) or descending (`DESC`); `LIMIT n` keeps only the first *n* rows.

- `SELECT fullName, feesPaid FROM STUDENT`
- `ORDER BY feesPaid DESC LIMIT 3;`

This answers: "Who are the three students who have paid the most?"

3.6 Aggregate Functions, GROUP BY and HAVING

Often the question is not about individual rows but about summaries: totals, averages, counts. SQL provides the **aggregate functions** `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. Used with `GROUP BY`, they compute one summary per group; `HAVING` then filters the groups.

- `SELECT classID, COUNT(*) AS nbStudents, AVG(feesPaid) AS avgFees`
- `FROM STUDENT`
- `GROUP BY classID`
- `HAVING COUNT(*) > 20;`

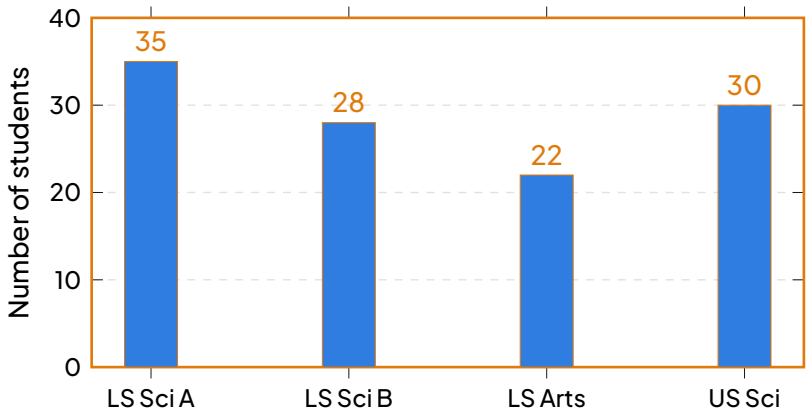
Reading: group the students by class; for each class, count the students and average the fees; keep only classes with more than 20 students.

WARNING **WHERE before grouping, HAVING after**

`WHERE` filters individual rows before any grouping is done, and it cannot mention aggregate functions. `HAVING` filters groups after aggregation, and that is where conditions on `COUNT(*)`, `AVG(...)`, etc. belong. Writing `WHERE COUNT(*) > 20` is a classic GCE error.

EXEMPLE **From query result to chart**

Suppose the query `SELECT className, COUNT(*) FROM STUDENT GROUP BY className;` returns: LS Science A: 35, LS Science B: 28, LS Arts: 22, US Science: 30. We can visualise it immediately:



Bar chart of the aggregate query: number of students per class (`COUNT(*)` with `GROUP BY className`).

3.7 Multi-table Retrieval: JOINs

Normalisation split our data into several tables, so many questions require combining them. The join is performed on the **foreign key** that links the tables.

DEFINITION **INNER JOIN and LEFT JOIN**

`INNER JOIN` returns only the rows that have a match in both tables. `LEFT JOIN` returns all rows of the left table, with the matching data from the right table where it exists, and `NULL` where it does not.

- `SELECT s.fullName, c.className`
- `FROM STUDENT s INNER JOIN CLASS c ON s.classID = c.classID;`

The short names `s` and `c` are **table aliases**: they make queries shorter and are essential when a column name (like `classID`) exists in both tables.



Venn-style comparison of `INNER JOIN` (intersection only) and `LEFT JOIN` (whole left table plus matches).

A `LEFT JOIN` is perfect for questions like "list all students and their class name, including students not yet assigned to a class" — the unassigned ones simply show `NULL` for `className`.

METHOD **Translating an English question into SQL**

1. What do we want to see? → the columns after `SELECT`.
2. Where is the data? → the table(s) after `FROM`; if several tables, identify the common key and write the `JOIN ... ON`.
3. Which rows qualify? → the condition after `WHERE`.
4. Is the question about groups ("per class", "for each level")? → `GROUP BY` on the grouping column, aggregate in `SELECT`, group condition in `HAVING`.
5. Any order or top-*n*? → `ORDER BY` and `LIMIT`.

WORKED EXAMPLE 3 **From English to SQL, step by step**

Question: "Display the name and fees paid of every female student in the class 'LS Science A' who has paid more than 30 000 FCFA, in alphabetical order of names."

SOLUTION

Step 1 — what: name and fees → `SELECT s.fullName, s.feesPaid`.
Step 2 — where: two tables, `STUDENT` and `CLASS`; common key `classID` → `FROM STUDENT s INNER JOIN CLASS c ON s.classID = c.classID`.
Step 3 — which rows: `WHERE s.gender = 'F' AND c.className = 'LS Science A' AND s.feesPaid > 30000`.
Step 4: no grouping. Step 5: `ORDER BY s.fullName ASC`.
Final query:

```
SELECT s.fullName, s.feesPaid
FROM STUDENT s INNER JOIN CLASS c ON s.classID = c.classID
WHERE s.gender = 'F' AND c.className = 'LS Science A' AND s.feesPaid > 30000
ORDER BY s.fullName ASC;
```

Tracing on paper: given the rows (Amina Bello, F, 50000, class 1) and (John Eta, M, 45000, class 1) with class 1 = 'LS Science A', the `WHERE` keeps only Amina (John fails the gender test), so the output is a single row: `Amina Bello | 50000.00`.

WARNING **Exam tip: find the common key first**

In every GCE JOIN question, before writing anything, identify the foreign key that links the tables (here `STUDENT.classID → CLASS.classID`) and write the `ON` condition with it. A join on the wrong columns produces a meaningless Cartesian product — the most frequent cause of lost marks.

EXERCISE 3 **Predicting query output**

Given `STUDENT` = {(1,'Ngo Bassa','F',20000,1), (2,'Ali Musa','M',0,1), (3,'Fon Claudia','F',55000,2)} and `CLASS` = {(1,'LS Sci A'), (2,'LS Arts'), (3,'US Sci')}, write the output of:

a)
```
SELECT fullName FROM STUDENT WHERE feesPaid = 0;
```

b)
```
SELECT className, COUNT(*) FROM STUDENT s JOIN CLASS c ON s.classID = c.classID GROUP BY className;
```

c)
```
SELECT className FROM CLASS c LEFT JOIN STUDENT s ON s.classID = c.classID WHERE s.studentID IS NULL;
```

SOLUTION **Exercise 1**

a) `Ali Musa` (only row with `feesPaid = 0`).
b) `LS Sci A | 2` and `LS Arts | 1` — the INNER JOIN drops 'US Sci' (no students), then `GROUP BY` counts per class.
c) `US Sci` — the LEFT JOIN keeps all classes; 'US Sci' has `NULL` student columns, and the `WHERE` selects exactly that. This is the standard idiom for finding unmatched rows.

KEY POINTS **Key points**

- SQL is declarative: DDL defines structures (`CREATE TABLE` with types and constraints), DML handles data (`INSERT`, `UPDATE`, `DELETE`, `SELECT`).
- Never omit `WHERE` in `UPDATE`/`DELETE`; test missing values with `IS NULL`, never `= NULL`.
- `WHERE` filters rows before grouping; `HAVING` filters groups after aggregation.
- Joins combine tables through the foreign key: `INNER JOIN` keeps matches only, `LEFT JOIN` keeps all rows of the left table.
- Exam method: English question → SELECT (what), FROM/JOIN (where, on the common key), WHERE (which rows), GROUP BY/HAVING (per-group questions), ORDER BY/LIMIT (order, top-n).

4 HANDS-ON PROJECT: BUILDING & BINDING A COMPLETE DATABASE

In the previous section you learnt to interrogate a database with SQL: `SELECT`, `WHERE`, `JOIN`, `GROUP BY` and the aggregate functions. But a real information system is more than a set of queries typed by hand. It is a complete project: someone models the real world, builds the tables, loads data, and then connects the database to a **data-entry form** so that an ordinary user — a bursar, a secretary — can record information without ever writing SQL. This final section brings everything together in one guided project, **SchoolManager**, exactly in the style of a GCE practical assessment. We shall follow the five phases of the database project workflow, from the ER diagram to the bound form, justifying every design decision as we go.

4.1 The Project Brief: SchoolManager

EXEMPLE **Project brief**

The administration of GBHS Mvan wants a mini database called SchoolManager to manage:

- Students: each student has a unique matricule, a name, a date of birth, a gender, and belongs to exactly one class.
- Classes: e.g. Form 5A, Lower Sixth Science; each class has a level and a class teacher.
- Subjects: e.g. Computer Science, Physics; each student registers for several subjects, and each subject is taken by several students.
- Fee payments: the annual fee is paid in instalments; each payment records the student, the amount, the date and the term (1, 2 or 3).

Required outputs: the list of **fees defaulters**, the size of each class, the total collected per term, and the number of students per subject.

METHOD **The 5-phase database project workflow**

Every database project, from a school project to a banking system, follows the same disciplined workflow:

1. Model: draw the ER diagram (entities, attributes, relationships, cardinalities) and validate it with users.
2. Normalise: derive the relational schema and refine it to 3NF; write the data dictionary.
3. Implement: create the tables in a DBMS with `CREATE TABLE`, including all constraints; insert sample data.

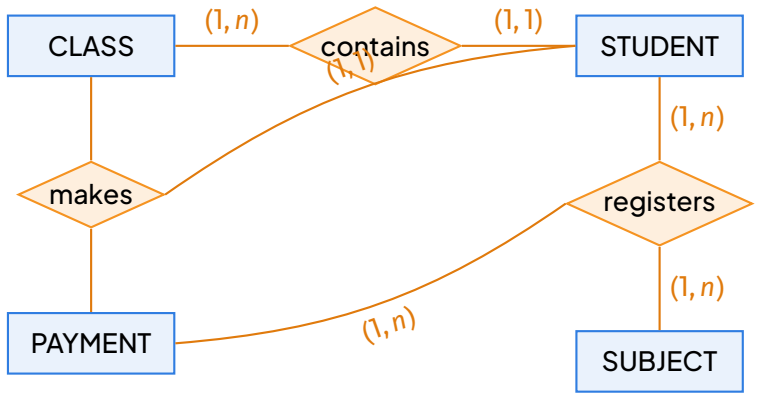
4. Query: write and test the retrieval queries that answer the brief.
5. Bind: connect a data-entry form to the tables, test insertions, and verify integrity.

Skipping a phase — especially phases 1 and 2 — is the classical cause of databases that must be rebuilt from scratch.

4.2 Phase 1: The ER Diagram and Its Validation

Working in groups, we identify four **entities** (`STUDENT`, `CLASS`, `SUBJECT`, `PAYMENT`) and three **relationships**:

- `CLASS` contains `STUDENT`: one class contains many students; each student belongs to one class ⇒ cardinality (1, *n*) on the student side, (1, 1) on the class side.
- `STUDENT` registers for `SUBJECT`: many-to-many (*n*, *n*) — a student takes several subjects, a subject has several students.
- `STUDENT` makes `PAYMENT`: one-to-many (1, *n*) — a student makes several payments; each payment concerns exactly one student.



ER diagram of SchoolManager: four entities and three relationships with their cardinalities.

The validation step consists of reading the diagram aloud as sentences ("one student makes many payments; one payment is made by exactly one student") and checking with the bursar that these sentences are true. A cardinality error discovered here costs five minutes; discovered after implementation, it costs days.

4.3 Phase 2: Normalised Schema and Data Dictionary

Applying the derivation rules of Section 2: each entity becomes a table; each (1, *n*) relationship pushes the "one" side's primary key into the "many" side as a **foreign key**; the (*n*, *n*) relationship `registers` becomes its own table, `ENROLMENT`, whose primary key is the pair of foreign keys. The schema is already in **3NF**: every non-key attribute depends on the key, the whole key, and nothing but the key. For example, in `STUDENT`, the attributes `fullName`, `dob`, `gender` and `classID`

depend only on `matricule`; there is no partial dependency (the key is simple) and no transitive dependency (no non-key attribute determines another).

DEFINITION **Data dictionary**

A **data dictionary** is the reference document of a database project that describes, for every table, each column with its name, data type, size, constraints (primary key, foreign key, NOT NULL, UNIQUE, CHECK) and meaning. It is the contract between the designer, the programmer and the examiner: anyone can rebuild or verify the database from the data dictionary alone.

Table	Column	Type	Constraint / meaning
CLASS	classID	INTEGER	PRIMARY KEY
CLASS	className	VARCHAR(20)	NOT NULL, UNIQUE (e.g. ‘L6 Science’)
CLASS	level	VARCHAR(15)	NOT NULL
STUDENT	matricule	VARCHAR(10)	PRIMARY KEY (e.g. ‘GBH2025-01’)
STUDENT	fullName	VARCHAR(60)	NOT NULL
STUDENT	dob	DATE	NOT NULL
STUDENT	gender	CHAR(1)	CHECK (gender IN (‘M’,‘F’))
STUDENT	classID	INTEGER	FOREIGN KEY → CLASS
SUBJECT	subjectID	INTEGER	PRIMARY KEY
SUBJECT	subjectName	VARCHAR(40)	NOT NULL, UNIQUE
ENROLMENT	matricule	VARCHAR(10)	PK, FK → STUDENT
ENROLMENT	subjectID	INTEGER	PK, FK → SUBJECT
PAYMENT	paymentID	INTEGER	PRIMARY KEY
PAYMENT	matricule	VARCHAR(10)	FOREIGN KEY → STUDENT
PAYMENT	amount	INTEGER	CHECK (amount > 0), in FCFA
PAYMENT	payDate	DATE	NOT NULL
PAYMENT	term	INTEGER	CHECK (term BETWEEN 1 AND 3)

4.4 Phase 3: Implementation in a DBMS

We now translate the data dictionary into SQL, here in a dialect understood by both SQLite and MySQL. Note how every constraint of the dictionary appears in the script: the data dictionary and the script must say exactly the same thing.

```
CREATE TABLE Class (
  classID INTEGER PRIMARY KEY,
  className VARCHAR(20) NOT NULL UNIQUE,
  level VARCHAR(15) NOT NULL);
```

```
CREATE TABLE Student (
  matricule VARCHAR(10) PRIMARY KEY,
  fullName VARCHAR(60) NOT NULL,
  dob DATE NOT NULL,
  gender CHAR(1) CHECK (gender IN ('M','F')),
  classID INTEGER REFERENCES Class(classID));
CREATE TABLE Subject (
  subjectID INTEGER PRIMARY KEY,
  subjectName VARCHAR(40) NOT NULL UNIQUE);
CREATE TABLE Enrolment (
  matricule VARCHAR(10) REFERENCES Student(matricule),
  subjectID INTEGER REFERENCES Subject(subjectID),
  PRIMARY KEY (matricule, subjectID));
CREATE TABLE Payment (
  paymentID INTEGER PRIMARY KEY,
  matricule VARCHAR(10) REFERENCES Student(matricule),
  amount INTEGER CHECK (amount > 0),
  payDate DATE NOT NULL,
  term INTEGER CHECK (term BETWEEN 1 AND 3));
```

WARNING **Creation order matters**

Because `Student` references `Class`, the `Class` table must be created before `Student`, and populated before students are inserted. Likewise `Enrolment` and `Payment` come last. Creating or filling tables in the wrong order causes foreign key errors — a very common practical-exam mistake.

Sample data (realistic, Cameroonian):

```
INSERT INTO Class VALUES (1,'L6 Science','Upper Sixth');
INSERT INTO Student VALUES ('GBH01','Amina Bello','2007-03-14','F',1);
INSERT INTO Student VALUES ('GBH02','Tabi Ernest','2006-11-02','M',1);
INSERT INTO Subject VALUES (1,'Computer Science');
INSERT INTO Enrolment VALUES ('GBH01',1);
```

```
INSERT INTO Payment VALUES (1,'GBH01',25000,'2024-09-20',1);
```

4.5 Phase 4: The Required Retrieval Queries

Each query below answers one requirement of the brief. Read each one and identify the role of every clause before running it.

Class size per class (JOIN + GROUP BY). The `LEFT JOIN` guarantees that even an empty class appears, with size 0:

```
SELECT c.className, COUNT(s.matricule) AS size
FROM Class c LEFT JOIN Student s ON s.classID = c.classID
GROUP BY c.className;
```

Total collected per term:

```
SELECT term, SUM(amount) AS totalFCFA
FROM Payment GROUP BY term ORDER BY term;
```

Students per subject:

```
SELECT sub.subjectName, COUNT(e.matricule) AS nbStudents
FROM Subject sub JOIN Enrolment e
  ON e.subjectID = sub.subjectID
GROUP BY sub.subjectName;
```

Fees defaulters (annual fee = 75 000 FCFA): students whose total payments fall below the fee. We group payments per student, sum them, and keep only the groups whose sum is below 75 000 — a condition on a group, hence `HAVING`, not `WHERE`:

```
SELECT s.matricule, s.fullName,
  75000 - IFNULL(SUM(p.amount),0) AS balance
FROM Student s LEFT JOIN Payment p
  ON p.matricule = s.matricule
GROUP BY s.matricule, s.fullName
HAVING IFNULL(SUM(p.amount),0) < 75000;
```

With our sample data, Amina has paid 25 000 FCFA (balance 50 000) and Tabi has paid nothing (balance 75 000): both appear in the defaulters list.

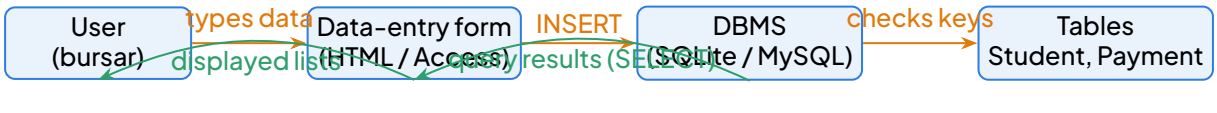
WARNING **LEFT JOIN and IFNULL for defaulters**

A defaulter may have made no payment at all. An inner `JOIN` would silently drop such a student — the worst possible defaulter disappears from the list! Use `LEFT JOIN` and replace

the resulting `NULL` sum by 0 with `IFNULL` (MySQL/SQLite) or `COALESCE` (standard SQL). This is a classic GCE trap.

4.6 Phase 5: Binding a Data-Entry Form

The final phase connects the database to its users. The architecture is always the same: the user types into a **form**; the form sends the data to the **DBMS**; the DBMS checks the constraints and stores the row in the tables; query results flow back to the user.



Architecture of the bound application: data flows from the form to the tables; results flow back.

With **MS Access**, the binding is automatic: create a form on the `Student` table with the Form Wizard; Access generates the `INSERT` for you. With **HTML + a script** (e.g. PHP or Python), the form collects the fields and the script executes a parameterised `INSERT`:

- `INSERT INTO Student VALUES (?, ?, ?, ?, ?);`

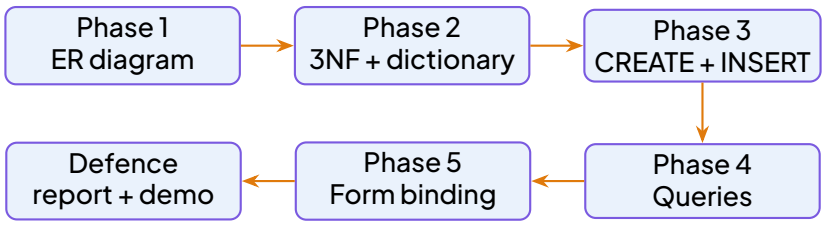
where each `?` is filled with one form field. Parameterised queries also protect the database against **SQL injection**, where a malicious user types SQL code into a form field. After each test insertion, run `SELECT * FROM Student;` to see the new row — visible proof that the binding works.

WARNING **Test integrity with deliberately wrong data**

A binding is only trustworthy if it rejects bad data. During testing, deliberately submit: a duplicate matricule (must fail: primary key violation), an empty name (must fail: NOT NULL), a gender 'X' (must fail: CHECK), a payment for a non-existent student (must fail: foreign key), a negative amount (must fail: CHECK). If any of these is accepted, the constraint is missing — fix the table, not the user!

4.7 Project Defence and Assessment

The GCE practical-style defence requires you to present, justify and demonstrate: show the ER diagram, explain why `ENROLMENT` exists (the many-to-many rule), justify 3NF (no transitive or partial dependencies), then run the four queries live and comment on the outputs.



Flow of the SchoolManager project, from modelling to the final defence.

KEY POINTS **Exam tip: what a project report must contain**

A complete project report includes: (1) the **ER diagram** with cardinalities; (2) the **normalised relational schema** with justification of 3NF; (3) the **data dictionary**; (4) the `CREATE TABLE` scripts with constraints; (5) the **sample queries** together with their actual outputs; (6) evidence of integrity testing. A report showing only screenshots of tables, without the schema and the queries, loses most of the marks.

DID YOU KNOW? **GCE extra: typical practical questions**

Two questions recur constantly in practical papers: (a) "Write the `CREATE TABLE` statement for `PAYMENT`, including the primary key, the foreign key and a check that the amount is positive"; (b) "Using your tables, write a query with a `JOIN` and `GROUP BY` giving the number of students per class." Master these two patterns and you secure the core of the practical marks.

WORKED EXAMPLE 4 **Worked exam-style task**

Statement. (a) Write the `CREATE TABLE` for `ENROLMENT`. (b) Write a query listing, for each class, the names of its students, ordered by class then by name.

SOLUTION

Solution. (a) The table has a composite primary key made of two foreign keys:

```
CREATE TABLE Enrolment (
  matricule VARCHAR(10) REFERENCES Student(matricule),
  subjectID INTEGER REFERENCES Subject(subjectID),
  PRIMARY KEY (matricule, subjectID));
```

(b) A simple join with ordering:

- `SELECT c.className, s.fullName`
- `FROM Class c JOIN Student s ON s.classID = c.classID`
- `ORDER BY c.className, s.fullName;`

Output (with our sample data): `L6 Science | Amina Bello`, **then** `L6 Science | Tabi Ernest`.

EXERCISE 4 **Mini-project extension**

Extend SchoolManager with a `TEACHER` table and a relationship "a teacher teaches a subject to a class". (1) Add the entity and relationship to the ER diagram with cardinalities. (2) Derive the new table(s), update the data dictionary, and write the `CREATE TABLE` script. (3) Write a query giving the subjects taught by each teacher.

SOLUTION **Exercise 1**

(1) `TEACHER(teacherID, teacherName)`; relationship `TEACHES` between `TEACHER`, `SUBJECT` and `CLASS`: a teacher teaches several subjects in several classes, so it is many-to-many, becoming a table. (2) `CREATE TABLE Teaches (teacherID INTEGER REFERENCES Teacher(teacherID), subjectID INTEGER REFERENCES Subject(subjectID), classID INTEGER REFERENCES Class(classID), PRIMARY KEY (teacherID, subjectID, classID));` — the composite key prevents duplicate assignments. (3) `SELECT t.teacherName, s.subjectName FROM Teacher t JOIN Teaches x ON x.teacherID = t.teacherID JOIN Subject s ON s.subjectID = x.subjectID ORDER BY t.teacherName;`

5 INTEGRATION ACTIVITY

6 DATABASE SYSTEMS & INFORMATION RETRIEVAL

6.1 From Flat Files to Relational Modelling (ER Diagrams)

EXPERIMENT **The messy spreadsheet at GBHS Bamenda**

The bursar of GBHS Bamenda (Government Bilingual High School) currently tracks school fees in a single Excel workbook named `Fees_Raw.xlsx`. The workbook contains one sheet with the following columns (a sample of 12 rows is shown; the real file has over 500 rows):

StudentID	StudentName	Class	FeeType	AmountDue	AmountPaid	DatePaid	ReceiptNo
S001	Ayuk John	Form 4A	Tuition	120000	120000	2024-09-05	R1001
S001	Ayuk John	Form 4A	PTA	15000	15000	2024-09-05	R1002
S002	Bih Grace	Form 4A	Tuition	120000	60000	2024-09-06	R1003
S002	Bih Grace	Form 4A	PTA	15000	0	NULL	NULL
S003	Chia Peter	Form 5B	Tuition	130000	130000	2024-09-07	R1004
S003	Chia Peter	Form 5B	Exam	20000	20000	2024-09-07	R1005
S004	Dora Nkem	Form 5B	Tuition	130000	130000	2024-09-08	R1006
S004	Dora Nkem	Form 5B	PTA	15000	15000	2024-09-08	R1007
S005	Epie Paul	Form 4A	Tuition	120000	120000	2024-09-09	R1008
S005	Epie Paul	Form 4A	PTA	15000	15000	2024-09-09	R1009
S005	Epie Paul	Form 4A	Exam	20000	0	NULL	NULL
S006	Fon Mary	Form 6S	Tuition	150000	75000	2024-09-10	R1010

Observed anomalies

- Redundancy: Student name and class repeat for every fee type.
- Update anomaly: If a student changes class, many rows must be edited.
- Insertion anomaly: A new fee type (e.g. `Sport`) cannot be added without a student.
- Deletion anomaly: Deleting the last payment of a student erases the student's identity.
- Inconsistent data: `ReceiptNo` is `NULL` when `AmountPaid=0`, but sometimes a receipt exists with zero payment (data entry error).

The bursar needs a reliable system that can:

1. Store each student once, each class once, each fee type once.
2. Record every payment with a unique receipt.
3. Answer instantly: defaulters list, total collected per class, outstanding balance per student, payments per fee type, receipts for a given student.
4. Allow the secretary to register a new payment through a simple web form.

DEFINITION **Entity–Relationship (ER) Model**

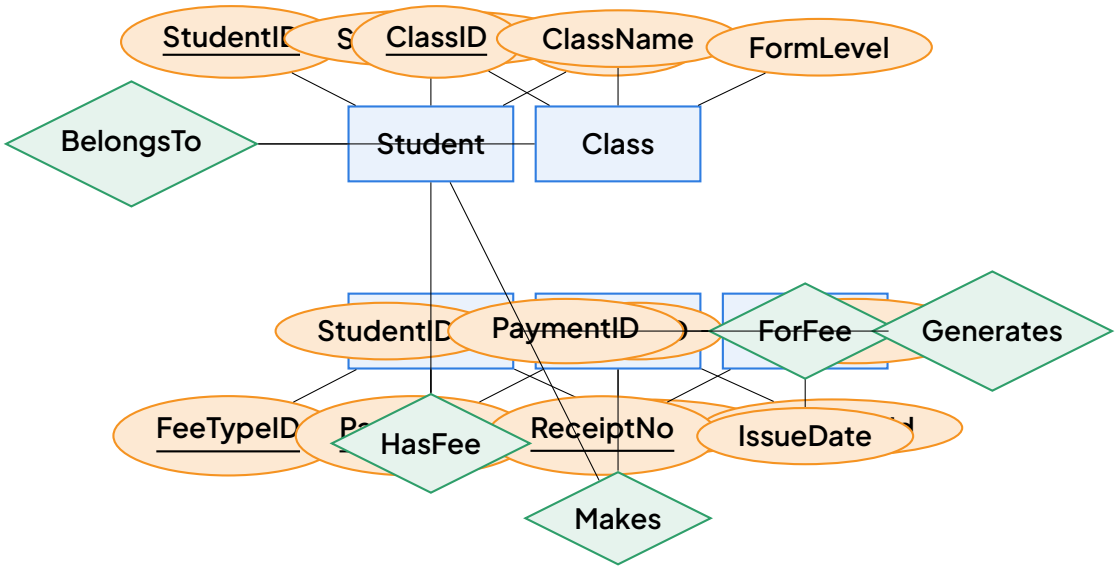
The ER model is a high-level conceptual data model that describes the structure of a database in terms of **entities** (objects of interest), **attributes** (properties of entities), and **relationships** (associations between entities). Chen notation uses rectangles for entities, ovals for attributes, diamonds for relationships, and lines to connect them. Primary keys are underlined; foreign keys are marked `FK`.

PROPERTY **Relationship Cardinalities**

Cardinality expresses the number of instances of one entity that can be associated with instances of another:

- 1:1 (One-to-One) — e.g. `Payment` – `Receipt`
- 1:N (One-to-Many) — e.g. `Class` – `Student`
- M:N (Many-to-Many) — resolved by an associative entity (e.g. `Student` – `FeeType` via `Payment`)

EXEMPLE **ER Diagram for GBHS Bamenda Fees System**



ER diagram for GBHS Bamenda fees system (Chen notation). All entities have strong primary keys; no weak entities.

KEY POINTS **Key Design Decisions**

- Class is separate from `Student` to avoid update anomalies when a student changes class.
- FeeType is linked to `Class` because the amount due (e.g. Tuition) differs by form level (Form 4 vs Form 6).
- Payment is the associative entity resolving the M:N between `Student` and `FeeType`; it records actual payments.
- Receipt is a 1:1 extension of `Payment` for audit trail (unique receipt number, issue date).

6.2 From ER Diagram to Normalised Relational Schema

DEFINITION **Functional Dependency (FD)**

Let R be a relation schema and $X, Y \subseteq R$. A functional dependency $X \rightarrow Y$ holds if, for any two tuples t_1, t_2 in any legal instance of R, $t_1[X] = t_2[X]$ implies $t_1[Y] = t_2[Y]$. X is the **determi-**

nant; Y is the **dependent**.

DEFINITION **Normal Forms (Exam-Relevant Definitions)**

- 1NF: All attribute values are atomic (no repeating groups, no composite attributes).
- 2NF: In 1NF and no **partial dependency** (a non-prime attribute depends on part of a candidate key).
- 3NF: In 2NF and no **transitive dependency** (a non-prime attribute depends on another non-prime attribute). Equivalently: for every FD $X \to A$ where A is a single attribute, either X is a superkey or A is prime (part of some candidate key).
- BCNF: For every FD $X \to Y$, X is a superkey. (Stronger than 3NF; examinable for distinction.)

METHOD **Normalisation Procedure: Universal Relation → 3NF**

1. Write the **universal relation** U containing every attribute from the flat file.
2. List all **functional dependencies** (FDs) derived from the business rules.
3. Find all **candidate keys** of U (minimal superkeys).
4. Check 1NF: ensure atomicity.
5. Check 2NF: if any candidate key is composite, remove partial dependencies.
6. Check 3NF: remove transitive dependencies by creating new relations for each determinant that is not a superkey.
7. Verify lossless-join and dependency preservation.

EXEMPLE **Step-by-Step Normalisation to 3NF**

1. Universal Relation

U(`StudentID, StudentName, ClassID, ClassName, FormLevel, FeeTypeID, FeeName, AmountDue, Paymen`

2. Functional Dependencies (from ER diagram and business rules)

```
StudentID → StudentName, ClassID
  ClassID → ClassName, FormLevel
FeeTypeID → FeeName, AmountDue, ClassID
PaymentID → StudentID, FeeTypeID, AmountPaid, DatePaid
ReceiptNo → PaymentID, IssueDate
```

3. Candidate Keys
Each determinant above is a candidate key for its own entity set. For the universal relation U, the only candidate key is the composite {StudentID, FeeTypeID, PaymentID} (or similar), but decomposition by entity is more natural. We proceed by synthesising relations from the FDs.
4. 1NF
The spreadsheet already has atomic values (no repeating groups). Satisfied.
5. 2NF
All candidate keys in the decomposed relations are single attributes (StudentID, ClassID, FeeTypeID, PaymentID, ReceiptNo). Hence no partial dependencies. Satisfied.
6. 3NF — Remove Transitive Dependencies

- StudentName, ClassID depend on StudentID (PK) → keep in Student.
- ClassName, FormLevel depend on ClassID (non-key in Student) → transitive. Create Class(ClassID, ClassName, FormLevel).
- FeeName, AmountDue, ClassID depend on FeeTypeID (PK) → keep in FeeType.
- StudentID, FeeTypeID, AmountPaid, DatePaid depend on PaymentID (PK) → keep in Payment.
- PaymentID, IssueDate depend on ReceiptNo (PK) → keep in Receipt.

PROPERTY **Final 3NF Relations (with PK/FK)**

Relation	Attributes (PK underlined, FK italic)
Class	ClassID, ClassName, FormLevel
Student	StudentID, StudentName, ClassID
FeeType	FeeTypeID, FeeName, AmountDue, ClassID
Payment	PaymentID, StudentID, FeeTypeID, AmountPaid, DatePaid
Receipt	ReceiptNo, PaymentID, IssueDate

- Student.ClassID → Class.ClassID

- `FeeType.ClassID` → `Class.ClassID`
- `Payment.StudentID` → `Student.StudentID`
- `Payment.FeeTypeID` → `FeeType.FeeTypeID`
- `Receipt.PaymentID` → `Payment.PaymentID` (also `UNIQUE` to enforce 1:1)

WARNING **Common Normalisation Mistakes**

- Forgetting that `FeeType` depends on `Class` (different amounts per form) → leads to update anomalies if merged into `Student`.
- Treating `Receipt` as an attribute of `Payment` instead of a separate entity → loses ability to re-issue or void receipts independently.
- Missing the `UNIQUE` constraint on `Receipt.PaymentID` → allows multiple receipts for one payment.

6.3 Querying & Information Retrieval with SQL

DEFINITION **SQL — Structured Query Language**

SQL is the standard language for relational database management. It comprises:

- DDL (Data Definition Language): `CREATE`, `ALTER`, `DROP`
- DML (Data Manipulation Language): `INSERT`, `UPDATE`, `DELETE`, `SELECT`
- DCL (Data Control Language): `GRANT`, `REVOKE`
- TCL (Transaction Control Language): `COMMIT`, `ROLLBACK`

GCE Advanced Level expects proficiency in `SELECT` with joins, aggregation, grouping, subqueries, and basic DDL/DML.

METHOD **Writing a `SELECT` Query — Step by Step**

1. Identify the output columns required.
2. List the tables needed and how they join (PK/FK).
3. Write the `FROM` clause with explicit `JOIN` syntax (prefer `INNER JOIN` / `LEFT JOIN`).

4. Add `WHERE` for row filtering before grouping.
5. Add `GROUP BY` for aggregation; every non-aggregated column in `SELECT` must appear in `GROUP BY`.
6. Add `HAVING` for filtering after aggregation.
7. Add `ORDER BY` for presentation.

WORKED EXAMPLE 5 **SQL Implementation (SQLite) — DDL & Sample Data**

1. CREATE TABLE Statements (with all integrity constraints)

METHOD **DDL Script**

```
PRAGMA foreign_keys = ON;   % Enable FK enforcement in SQLite
CREATE TABLE Class (
 ClassID TEXT PRIMARY KEY,
 ClassName TEXT NOT NULL,
 FormLevel TEXT NOT NULL
);
CREATE TABLE Student (
 StudentID TEXT PRIMARY KEY,
 StudentName TEXT NOT NULL,
 ClassID TEXT NOT NULL,
 FOREIGN KEY (ClassID) REFERENCES Class(ClassID) ON UPDATE CASCADE ON
DELETE RESTRICT
);
CREATE TABLE FeeType (
 FeeTypeID TEXT PRIMARY KEY,
 FeeName TEXT NOT NULL,
 AmountDue INTEGER NOT NULL CHECK (AmountDue >= 0),
 ClassID TEXT NOT NULL,
 FOREIGN KEY (ClassID) REFERENCES Class(ClassID) ON UPDATE CASCADE ON
DELETE RESTRICT
);
CREATE TABLE Payment (
 PaymentID INTEGER PRIMARY KEY AUTOINCREMENT,
 StudentID TEXT NOT NULL,
 FeeTypeID TEXT NOT NULL,
 AmountPaid INTEGER NOT NULL CHECK (AmountPaid >= 0),
 DatePaid DATE NOT NULL,
 FOREIGN KEY (StudentID) REFERENCES Student(StudentID) ON UPDATE
CASCADE ON DELETE RESTRICT,
 FOREIGN KEY (FeeTypeID) REFERENCES FeeType(FeeTypeID) ON UPDATE
CASCADE ON DELETE RESTRICT
);
```

2. Sample Data Loading (matching the spreadsheet)

METHOD **DML Script**

```
-- Classes
INSERT INTO Class VALUES ('C4A','Form 4A','4'), ('C5B','Form 5B','5'), ('C6S','Form 6S','6');
-- Students
INSERT INTO Student VALUES ('S001','Ayuk John','C4A'), ('S002','Bih Grace','C4A'), ('S003','Chia Peter','C5B'), ('S004','Dora Nkem','C5B'), ('S005','Epie Paul','C4A'), ('S006','Fon Mary','C6S');
-- Fee Types (note: same FeeName, different ClassID and AmountDue)
INSERT INTO FeeType VALUES ('FT1','Tuition',120000,'C4A'), ('FT2','PTA',15000,'C4A'), ('FT3','Exam',20000,'C4A'),
 ('FT4','Tuition',130000,'C5B'), ('FT5','PTA',15000,'C5B'), ('FT6','Exam',20000,'C5B'),
 ('FT7','Tuition',150000,'C6S');
-- Payments (PaymentID auto-assigned: 1..12)
INSERT INTO Payment (StudentID,FeeTypeID,AmountPaid,DatePaid) VALUES
 ('S001','FT1',120000,'2024-09-05'), ('S001','FT2',15000,'2024-09-05'),
 ('S002','FT1',60000,'2024-09-06'), ('S002','FT2',0,'2024-09-06'),
 ('S003','FT4',130000,'2024-09-07'), ('S003','FT6',20000,'2024-09-07'),
 ('S004','FT4',130000,'2024-09-08'), ('S004','FT5',15000,'2024-09-08'),
 ('S005','FT1',120000,'2024-09-09'), ('S005','FT2',15000,'2024-09-09'),
 ('S005','FT3',0,'2024-09-09'), ('S006','FT7',75000,'2024-09-10');
-- Receipts (linked 1:1 to Payment)
INSERT INTO Receipt VALUES ('R1001',1,'2024-09-05'), ('R1002',2,'2024-09-05'), ('R1003',3,'2024-09-06'),
 ('R1004',4,'2024-09-07'), ('R1005',5,'2024-09-07'), ('R1006',6,'2024-09-08'),
 ('R1007',7,'2024-09-08'), ('R1008',8,'2024-09-09'), ('R1009',9,'2024-09-09'), ('R1010',10,'2024-09-10');
```

WORKED EXAMPLE 6 **Information Retrieval — Five Required Queries**

Query 1: Defaulters on Tuition (current academic year)
Logic: For each student, find the Tuition fee type for their class, sum payments, compare with AmountDue.

METHOD **SQL**

```
SELECT s.StudentID, s.StudentName, c.ClassName, ft.FeeName,
 ft.AmountDue - COALESCE(SUM(p.AmountPaid),0) AS Balance
FROM Student s
JOIN Class c ON s.ClassID = c.ClassID
JOIN FeeType ft ON ft.ClassID = c.ClassID AND ft.FeeName = 'Tuition'
LEFT JOIN Payment p ON p.StudentID = s.StudentID AND p.FeeTypeID =
ft.FeeTypeID
GROUP BY s.StudentID, ft.FeeTypeID
HAVING Balance > 0;
```

Expected output (sample): S002 (Bih Grace) owes 60,000 FCFA; S006 (Fon Mary) owes 75,000 FCFA.
Query 2: Total Amount Collected per Class

METHOD **SQL**

```
SELECT c.ClassName, SUM(p.AmountPaid) AS TotalCollected
FROM Payment p
JOIN Student s ON p.StudentID = s.StudentID
JOIN Class c ON s.ClassID = c.ClassID
GROUP BY c.ClassID, c.ClassName
ORDER BY TotalCollected DESC;
```

Query 3: Total Amount Paid per Fee Type (whole school)

METHOD **SQL**

```
SELECT ft.FeeName, SUM(p.AmountPaid) AS TotalPaid
FROM Payment p
JOIN FeeType ft ON p.FeeTypeID = ft.FeeTypeID
GROUP BY ft.FeeTypeID, ft.FeeName
ORDER BY TotalPaid DESC;
```

Query 4: All Receipts for a Given Student (e.g. S001)

METHOD **SQL**

```
SELECT r.ReceiptNo, r.IssueDate, ft.FeeName, p.AmountPaid
FROM Receipt r
JOIN Payment p ON r.PaymentID = p.PaymentID
JOIN FeeType ft ON p.FeeTypeID = ft.FeeTypeID
WHERE p.StudentID = 'S001'
ORDER BY r.IssueDate;
```

Query 5: Outstanding Balance per Student (all fee types)

METHOD **SQL**

```
SELECT s.StudentID, s.StudentName, c.ClassName,
 SUM(ft.AmountDue) - COALESCE(SUM(p.AmountPaid),0) AS Outstanding
FROM Student s
JOIN Class c ON s.ClassID = c.ClassID
JOIN FeeType ft ON ft.ClassID = c.ClassID
LEFT JOIN Payment p ON p.StudentID = s.StudentID AND p.FeeTypeID =
ft.FeeTypeID
GROUP BY s.StudentID, s.StudentName, c.ClassName
ORDER BY Outstanding DESC;
```

Note: *COALESCE* handles students with no payments yet (NULL sum).

WARNING **SQL Traps for GCE Candidates**

- Using `WHERE` instead of `HAVING` for aggregate conditions.
- Forgetting `LEFT JOIN` when the "many" side may be empty (e.g. students with no payments).
- Omitting non-aggregated columns from `GROUP BY` (causes error in strict SQL modes).
- Writing implicit joins (comma-separated tables) instead of explicit `JOIN` syntax.
- Not qualifying column names with table aliases when the same column name exists in multiple tables.

6.4 Hands-on Project: Building & Binding a Complete Database

EXPERIMENT **Project Specification: GBHS Bamenda Fees Management System**

Objective: Implement the complete database cycle: ER modelling → 3NF schema → SQL DDL/DML → Querying → Web form binding.
Deliverables (Team of 3–4 students):

1. ER diagram (Chen notation) with cardinalities, PK/FK, weak entities if any.
2. Normalisation report: universal relation, FDs, candidate keys, 1NF/2NF/3NF steps, final relations.
3. SQLite database file (`gbhs_fees.db`) with all tables, constraints, and sample data.
4. Five SQL query files (`query1.sql` … `query5.sql`) with comments.
5. A minimal web application (Python Flask + SQLite) that allows the secretary to register a payment.
6. 5-minute presentation + live demo (assessment rubric: 20 marks).

METHOD **Minimal Flask Application — Payment Registration Form**

File: `app.py`

```
from flask import Flask, request, render_template_string, redirect, url_for, flash
import sqlite3, os
app = Flask(__name__)
app.secret_key = 'dev-key-change-in-production'
DATABASE = 'gbhs_fees.db'

def get_db():
 conn = sqlite3.connect(DATABASE)
 conn.row_factory = sqlite3.Row
 conn.execute('PRAGMA foreign_keys = ON')
 return conn

def init_db():
 with app.app_context():
 db = get_db()
 with open('schema.sql', 'r') as f:
 db.executescript(f.read())
```

```
    db.commit()

@app.route('/')
def index():
    return redirect(url_for('register_payment'))

@app.route('/pay', methods=['GET','POST'])
def register_payment():
    db = get_db()
    if request.method == 'POST':
        sid = request.form['student_id'].strip()
        ftid = request.form['fee_type_id'].strip()
        try:
            amt = int(request.form['amount_paid'])
            if amt < 0: raise ValueError
        except ValueError:
            flash('Amount must be a non-negative integer.', 'error')
            return redirect(url_for('register_payment'))
        dte = request.form['date_paid']
        try:
            cur = db.cursor()
            cur.execute('INSERT INTO Payment (StudentID,FeeTypeID,AmountPaid,DatePaid)
VALUES (?,?,?,?)',
                        (sid, ftid, amt, dte))
            pid = cur.lastrowid
            rno = 'R' + str(pid + 1000)
            cur.execute('INSERT INTO Receipt (ReceiptNo,PaymentID,IssueDate) VALUES
(?,?,?)',
                        (rno, pid, dte))
```

```
    db.commit()
    flash(f'Payment recorded. Receipt: {rno}', 'success')
except sqlite3.IntegrityError as e:
    db.rollback()
    flash(f'Database error: {e}', 'error')
finally:
    db.close()
    return redirect(url_for('register_payment'))

# GET: fetch students and fee types for dropdowns
students = db.execute('SELECT StudentID, StudentName FROM Student ORDER BY StudentName').fetchall()
feetypes = db.execute('SELECT FeeTypeID, FeeName, AmountDue FROM FeeType ORDER BY FeeName').fetchall()
db.close()
return render_template_string(FORM_HTML, students=students, feetypes=feetypes)

FORM_HTML = '''
<!DOCTYPE html><html><head><title>GBHS Fees Payment</title>
<style>body{font-family:Arial;margin:2rem; .form-group{margin-bottom:1rem;
label{display:block;margin-bottom:0.3rem; input,select{padding:0.5rem;width:100%;max-width:30
button{padding:0.7rem 1.5rem;background:#2c7;color:white;border:none;cursor:pointer;
.flash{padding:1rem;margin:1rem 0;border-radius:4px; .success{background:#d4edda;color:#15572
.error{background:#f8d7da;color:#721c24;}</style>
</head><body>
<h1>GBHS Bamenda --- Register Payment</h1>
{% with messages = get_flashed_messages(with_categories=true) %}
{% for cat, msg in messages %}
<div class="flash {{cat}}">{{ msg }}</div>
{% endfor %}
```

```
% endwith %
<form method="post">
<div class="form-group"><label>Student</label><select name="student_id"
required>
% for s in students %<option value="{{ s.StudentID }}">{{ s.StudentID }} -
{{ s.StudentName }}</option>% endfor %
</select></div>
<div class="form-group"><label>Fee Type</label><select name="fee_type_id"
required>
% for f in feetypes %<option value="{{ f.FeeTypeID }}">{{ f.FeeName }} ({{
f.AmountDue }} FCFA)</option>% endfor %
</select></div>
<div class="form-group"><label>Amount Paid (FCFA)</label><input
type="number" name="amount_paid" min="0" required></div>
<div class="form-group"><label>Date Paid</label><input type="date"
name="date_paid" required></div>
<button type="submit">Register Payment</button>
</form></body></html>'''

if __name__ == '__main__':
 if not os.path.exists(DATABASE): init_db()
 app.run(debug=True, host='0.0.0.0', port=5000)
```

KEY POINTS **How the Form Connects to the Database**

1. Connection string: In SQLite, it is simply the file path `'gbhs_fees.db'`. In MySQL/PostgreSQL it would be `'mysql://user:pass@host/db'`.
2. Parameterised queries: The `?` placeholders in `cur.execute('INSERT ... VALUES (?,?,?,?)', (sid,ftid,amt,dte))` prevent SQL injection. Never use string formatting (f-strings) for SQL values.
3. Transaction control: `db.commit()` makes changes permanent; `db.rollback()` undoes them on error.

4. Connection lifecycle: Open at start of request (`get_db()`), close at end (`db.close()`). In production, use a connection pool or Flask-SQLAlchemy.

DID YOU KNOW? **Cameroon Context: School Fees in Public Secondary Schools**

In Cameroon, public secondary schools (like GBHS) collect several fee types:

- Tuition (Frais de scolarité): Set by MINEDUB/MINESEC; varies by cycle (Form 1–5 vs Form 6) and stream (Arts/Science).
- PTA (Parent-Teacher Association): Fixed per school, approved by General Assembly.
- Exam fees (Frais d'examen): For official exams (GCE O/L, A/L, Probatoire, Baccalauréat).
- Sports/Insurance/Other: Optional or mandatory per school.

The bursar (intendant) is responsible for collection, receipting, and reporting to the Principal and the Divisional Delegate of Secondary Education. A computerised system reduces fraud, errors, and delays in reporting.

EXERCISE 5 **Practice Questions for GCE Revision**

1. Draw the ER diagram (Chen notation) for a library system with entities `Book`, `Member`, `Loan`, `Fine`. Indicate cardinalities and identify any weak entities.
2. Given the universal relation $R(A, B, C, D, E)$ with FDs $A \to B$, $B \to C$, $A \to D$, $D \to E$, find all candidate keys and normalise to 3NF.
3. Write a SQL query to find the second-highest `AmountPaid` in the `Payment` table (without using `LIMIT` / `TOP`).
4. Explain the difference between `INNER JOIN` and `LEFT JOIN` with a concrete example from the GBHS fees database.
5. In the Flask application, why is `PRAGMA foreign_keys = ON` necessary in SQLite? What happens if it is omitted?

SOLUTION **Exercise 1**

Entities: `Book`(ISBN, Title, Author, Year), `Member`(MemberID, Name, Email, JoinDate), `Loan`(LoanID, MemberID, ISBN, DateOut, DueDate, DateReturned), `Fine`(FineID, LoanID, Amount, PaidStatus).

Relationships: `Member` 1:N `Loan` (Borrows), `Book` 1:N `Loan` (IsLoaned), `Loan` 1:1 `Fine` (Incurs). No weak entities.

SOLUTION **Exercise 2**

FDs: $A \to B, B \to C, A \to D, D \to E$.
Candidate key: $\{A\}$ (since $A^+ = \{A, B, C, D, E\}$).
3NF Decomposition: $R_1(A, B, D)$, $R_2(B, C)$, $R_3(D, E)$. All in 3NF, lossless-join, dependency-preserving.

SOLUTION **Exercise 3**

```
SELECT MAX(AmountPaid) FROM Payment WHERE AmountPaid < (SELECT MAX(AmountPaid)
FROM Payment);
```

SOLUTION **Exercise 4**

`INNER JOIN` returns only matching rows (e.g. `Student` ⍰ `Payment` shows only students who have paid). `LEFT JOIN` returns all rows from the left table plus matches from right (e.g. `Student` ⍰ `Payment` shows all students, with `NULL` for those who never paid).

SOLUTION **Exercise 5**

SQLite disables foreign key enforcement by default for backward compatibility. Without `PRAGMA foreign_keys = ON`, `INSERT` into `Payment` with a non-existent `StudentID` would succeed, breaking referential integrity.

KEY POINTS **Presentation & Defence Checklist (5 minutes, 20 marks)**

- Correctness (8): ER diagram matches requirements; normalisation steps are valid; SQL runs without errors; queries return correct results; form inserts data correctly.
- Completeness (4): All 5 queries implemented; all tables have PK/FK/CHECK/NOT NULL; form handles validation and errors.
- Clarity (4): Slides are readable; SQL is formatted; ER diagram uses standard notation; team explains trade-offs (e.g. separate `Receipt` table vs. column in `Payment`).

- Teamwork (4): Each member speaks; smooth transitions; live demo works; answers questions confidently.

7 METHODS AND WORKED EXAMPLES

8 DATABASE SYSTEMS & INFORMATION RETRIEVAL

8.1 From Flat Files to Relational Modelling (ER Diagrams)

Skill 1: Designing an Entity–Relationship (ER) Diagram from Requirements

METHOD **Drawing an ER Diagram**

1. Identify entities: List all distinct real-world objects (e.g. Student, Book, Loan).
2. Determine attributes: For each entity, note simple, composite, multi-valued and derived attributes; underline the primary key.
3. Find relationships: Discover associations between entities (e.g. borrows between Student and Book). Specify cardinality (1:1, 1:N, M:N) and participation (total/partial).
4. Add weak entities: If an entity depends on another for identification (e.g. Loan identified by Student + Book + date), mark it with a double rectangle and a partial key.
5. Draw the diagram: Use rectangles for entities, diamonds for relationships, ovals for attributes, lines to connect. Indicate cardinality with "1" and "N" (or "M") on the lines.
6. Review and refine: Check for redundancy, ensure every relationship is necessary, and verify that all requirements are captured.

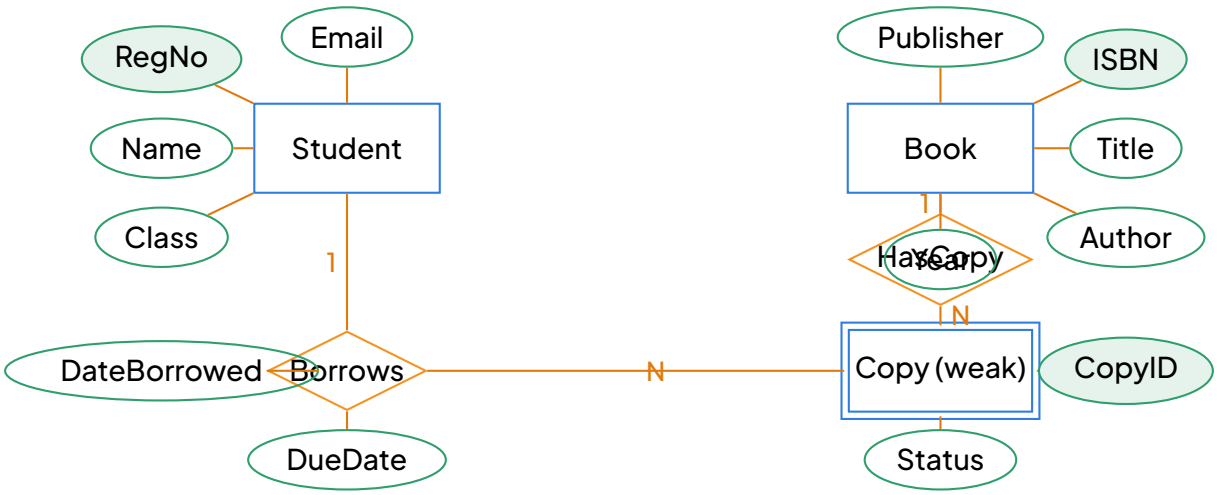
EXEMPLE **Library ER Diagram – Textual Description**

Scenario: A secondary school library in Yaoundé needs a database. Each Student has a registration number (RegNo), name, class and email. Each Book has an ISBN, title, author, publisher and year. A student may borrow many books; a book copy can be borrowed by at most one student at a time. The system must record the date_borrowed and due_date for each loan. A book may have several copies; each copy has a unique copy_id. Entities:

- `Student(RegNo, Name, Class, Email)` – strong entity, PK `RegNo`.
- `Book(ISBN, Title, Author, Publisher, Year)` – strong entity, PK `ISBN`.
- `Copy(CopyID, ISBN, Status)` – weak entity dependent on `Book`; partial key `CopyID`, composite PK (`ISBN`, `CopyID`).

Relationships:

- `Borrows` between `Student` and `Copy`: cardinality 1:N (one student borrows many copies; each copy borrowed by at most one student at a time). Participation: Student – partial, Copy – total (every loan involves a copy). Attributes: `DateBorrowed`, `DueDate`.
- `HasCopy` between `Book` and `Copy`: 1:N, total participation for Copy (a copy cannot exist without its book).



ER Diagram for the School Library System

WORKED EXAMPLE 7 **Library ER Diagram**

Statement: A secondary school library in Yaoundé needs a database. Each Student has a registration number (RegNo), name, class and email. Each Book has an ISBN, title, author, publisher and year. A student may borrow many books; a book copy can be borrowed by at most one student at a time. The system must record the date_borrowed and due_date for each loan. A book may have several copies; each copy has a unique copy_id. Draw the ER diagram.

SOLUTION

Solution:

1. Entities: `Student(RegNo, Name, Class, Email)`, `Book(ISBN, Title, Author, Publisher, Year)`, `Copy(CopyID, ISBN, Status)`. `Copy` is a weak entity dependent on `Book` (partial key `CopyID`).
2. Relationships:
   - `Borrows` between `Student` and `Copy`: cardinality 1:N (one student borrows many copies; each copy borrowed by at most one student). Participation: Student –

partial, Copy – total (every loan involves a copy).

- `HasCopy` between `Book` and `Copy`: 1:N, total participation for Copy (a copy cannot exist without its book).

3. Attributes on relationship: `Borrows` carries `DateBorrowed` and `DueDate`.
4. Diagram: See the figure above. Rectangles: `Student`, `Book`, `Copy` (double rectangle). Diamonds: `Borrows`, `HasCopy`. Ovals: attributes attached to respective entities/relationships; primary keys underlined. Lines: `Student` – `Borrows` – `Copy` (label "1" near Student, "N" near Copy). `Book` – `HasCopy` – `Copy` (label "1" near Book, "N" near Copy).

Key reflex: Always ask "Can this entity exist independently?" to spot weak entities. In exams, draw neatly with a ruler; label cardinalities clearly.

8.2 From ER Diagram to Normalised Relational Schema

Skill 2: Mapping an ER Diagram to a Normalised Relational Schema

METHOD **ER-to-Relational Mapping Rules**

1. Strong entity → Table: Create a relation with all simple attributes; primary key = entity's key.
2. Weak entity → Table: Include the partial key plus the primary key of the owner entity (foreign key). Composite primary key = (owner PK, partial key).
3. 1:N relationship: Add the primary key of the "1" side as a foreign key in the table of the "N" side. If the relationship has attributes, attach them to the "N" side table.
4. M:N relationship: Create a new junction table whose primary key is the combination of the two foreign keys; include any relationship attributes.
5. Multi-valued attribute: Create a separate table (entity PK + attribute) with a composite primary key.
6. Composite attribute: Decompose into its simple components in the same table.
7. Check normalisation: After mapping, verify each relation is at least in 3NF (no transitive dependencies).

WORKED EXAMPLE 8 **Relational Schema for the Library**

Statement: Using the ER diagram from Skill 1, produce the set of relational tables with primary keys, foreign keys and a brief justification of normalisation up to 3NF.

SOLUTION

Solution:

1. `Student`(RegNo, Name, Class, Email) – strong entity, 1NF (atomic attributes), no partial/transitive dependencies → 3NF.
2. `Book`(ISBN, Title, Author, Publisher, Year) – strong entity, 3NF.
3. `Copy`(ISBN, CopyID, Status) – weak entity; composite PK (ISBN, CopyID). ISBN is FK referencing `Book`. No non-key attribute depends on part of the key → 3NF.
4. `Borrows`(RegNo, ISBN, CopyID, DateBorrowed, DueDate) – junction table for the 1:N relationship with attributes. PK = (RegNo, ISBN, CopyID, DateBorrowed) to allow multiple loans of same copy by same student at different times. FKs: RegNo → Student, (ISBN, CopyID) → Copy. Non-key attribute DueDate depends on whole key → 3NF.
5. `HasCopy` is absorbed into `Copy` via the FK ISBN; no separate table needed.

Verification: Every non-key attribute is fully functionally dependent on the whole primary key (2NF) and no non-key attribute depends on another non-key attribute (3NF). The schema is ready for implementation.

Skill 3: Normalising a Relation to 3NF / BCNF

METHOD **Normalisation Procedure (up to BCNF)**

1. Start with a universal relation containing all attributes from the unnormalised form (e.g. a flat file).
2. Find all functional dependencies (FDs) from the requirements (e.g. RegNo → Name, Class; ISBN → Title, Author).
3. Compute candidate keys using attribute closure.
4. 1NF: Ensure atomic values (no repeating groups). If repeating groups exist, create separate relations.
5. 2NF: Remove partial dependencies (non-key attribute depends on part of a composite key). Decompose.
6. 3NF: Remove transitive dependencies (non-key attribute depends on another non-key attribute). Decompose.
7. BCNF: For every FD X → Y, X must be a superkey. If not, decompose using that FD.

8. Check lossless join and dependency preservation after each decomposition.

WORKED EXAMPLE 9 **Normalising a Student–Course–Lecturer Table**

Statement: A flat file contains: `RegNo`, `StudentName`, `Class`, `CourseCode`, `CourseTitle`, `LecturerID`, `LecturerName`, `LecturerOffice`, `Grade`. A student takes many courses; a course is taught by exactly one lecturer; a lecturer teaches many courses. A student receives one grade per course. Normalise to BCNF.

SOLUTION

Solution:

1. FDs identified:

$$\begin{aligned} \texttt{RegNo} &\rightarrow \texttt{StudentName, Class} \\ \texttt{CourseCode} &\rightarrow \texttt{CourseTitle, LecturerID} \\ \texttt{LecturerID} &\rightarrow \texttt{LecturerName, LecturerOffice} \\ \texttt{RegNo, CourseCode} &\rightarrow \texttt{Grade} \end{aligned}$$

2. Candidate key: {`RegNo, CourseCode`} (closure gives all attributes).
3. 1NF: Already atomic.
4. 2NF: Partial dependencies exist: `RegNo` → `StudentName, Class` and `CourseCode` → `CourseTitle, LecturerID`. Decompose:
   - `Student`(RegNo, StudentName, Class)
   - `Course`(CourseCode, CourseTitle, LecturerID)
   - `Lecturer`(LecturerID, LecturerName, LecturerOffice)
   - `Enrolment`(RegNo, CourseCode, Grade)
5. 3NF: Check transitive dependencies: In `Course`, `LecturerID` → `LecturerName, LecturerOffice` (non-key depends on non-key). Already separated into `Lecturer`. All relations now 3NF.
6. BCNF: Verify each FD's left side is a superkey in its relation:
   - `Student`: RegNo is PK → OK.
   - `Lecturer`: LecturerID is PK → OK.
   - `Course`: CourseCode is PK; LecturerID is FK, not a determinant of non-key attributes in this relation → OK.
   - `Enrolment`: (RegNo, CourseCode) is PK; Grade depends on whole key → OK.

All relations satisfy BCNF. The decomposition is lossless (common keys are PKs) and dependency preserving.

Exam tip: Always list FDs explicitly; missing an FD leads to an incomplete normalisation.

8.3 Querying & Information Retrieval with SQL

Skill 4: Writing Core SQL Queries (SELECT, JOIN, GROUP BY, HAVING)

METHOD **Constructing a SELECT Query**

1. Identify the target columns (use explicit names, avoid *).
2. Specify the FROM clause: List base tables; use aliases for brevity.
3. Add JOINs: INNER JOIN for matching rows, LEFT JOIN to keep all rows from the left table. Join on foreign-key = primary-key columns.
4. Filter rows: WHERE clause for conditions on columns (use AND, OR, IN, BETWEEN, LIKE).
5. Aggregate: GROUP BY non-aggregated columns; use COUNT, SUM, AVG, MIN, MAX.
6. Filter groups: HAVING clause with aggregate conditions.
7. Sort/limit: ORDER BY column [ASC|DESC]; LIMIT n (or TOP n in some dialects).
8. Test incrementally: Run the query step by step (FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY).

WORKED EXAMPLE 10 **Query: Books Borrowed per Student with Overdue Count**

Statement: Using the library schema (Student, Copy, Borrows), write a query that lists each student's registration number, name, total number of books currently borrowed, and the number of those that are overdue (due_date < today). Assume today is 2025-06-15. Show only students who have at least one overdue book.

SOLUTION

Solution:

```
SELECT s.RegNo,
  s.Name,
  COUNT(b.CopyID) AS TotalBorrowed,
  SUM(CASE WHEN b.DueDate < '2025-06-15' THEN 1 ELSE 0 END) AS OverdueCount
FROM Student s
JOIN Borrows b ON s.RegNo = b.RegNo
WHERE b.DateBorrowed <= '2025-06-15
GROUP BY s.RegNo, s.Name
HAVING SUM(CASE WHEN b.DueDate < '2025-06-15' THEN 1 ELSE 0 END) > 0
ORDER BY OverdueCount DESC;
```

Explanation:

- `JOIN` links students to their loans.
- `CASE` expression inside `SUM` counts overdue loans per student.
- `GROUP BY` includes all non-aggregated SELECT columns (RegNo, Name).
- `HAVING` filters groups after aggregation; only students with OverdueCount > 0 appear.
- `ORDER BY` sorts worst offenders first.

Common mistake: Putting the overdue condition in WHERE would discard rows before grouping, making the total count wrong. Use HAVING for aggregate filters.

Skill 5: Advanced SQL – Subqueries, Views, and Transactions

METHOD **Using Subqueries, Views and Transactions**

1. Subquery in WHERE: Use `IN`, `EXISTS`, or comparison operators. Correlated subqueries reference outer query columns.
2. Subquery in FROM (derived table): Wrap a SELECT in parentheses, give it an alias, then join it like a table.
3. CREATE VIEW: `CREATE VIEW view_name AS SELECT ...;` – stores a virtual table for reuse and security.
4. Transaction control: `START TRANSACTION;` ... `COMMIT;` or `ROLLBACK;`. Use for multi-step operations that must be atomic (e.g. borrowing a book: insert loan, update copy status).
5. Error handling: In application code, catch SQL exceptions and issue ROLLBACK.

WORKED EXAMPLE 11 **View for Overdue Loans & Transaction for Borrowing**

Statement:

1. Create a view `OverdueLoans` showing RegNo, StudentName, CopyID, BookTitle, DueDate for all loans where DueDate < CURRENT_DATE and the copy is still not returned (assume a `Returned` boolean in Borrows).
2. Write a transaction (in pseudocode) that a student (RegNo = 'S123') borrows copy (ISBN='978-0131103627', CopyID=5). Steps: check copy status = 'Available', insert a row in Borrows with DateBorrowed = today, DueDate = today + 14 days, set Copy.Status = 'On Loan'. If any step fails, roll back.

SOLUTION

Solution:

1. View:

```
CREATE VIEW OverdueLoans AS
SELECT s.RegNo,
  s.Name AS StudentName,
  b.CopyID,
  bk.Title AS BookTitle,
  b.DueDate
FROM Student s
JOIN Borrows b ON s.RegNo = b.RegNo
JOIN Copy c ON b.ISBN = c.ISBN AND b.CopyID = c.CopyID
JOIN Book bk ON c.ISBN = bk.ISBN
WHERE b.Returned = FALSE
  AND b.DueDate < CURRENT_DATE;
```

2. Transaction (pseudocode):

```
START TRANSACTION;
-- 1. Lock the copy row to avoid race conditions
SELECT Status FROM Copy
WHERE ISBN = '978-0131103627' AND CopyID = 5
FOR UPDATE;
-- 2. Check availability (application logic)
IF Status <> 'Available' THEN
  ROLLBACK;
  SIGNAL ERROR 'Copy not available';
END IF;
-- 3. Insert loan record
INSERT INTO Borrows (RegNo, ISBN, CopyID, DateBorrowed, DueDate, Returned)
VALUES ('S123', '978-0131103627', 5, CURRENT_DATE, DATE_ADD(CURRENT_DATE, INTERVAL 14 DAY
-- 4. Update copy status
UPDATE Copy
SET Status = 'On Loan'
WHERE ISBN = '978-0131103627' AND CopyID = 5;
COMMIT;
```

Notes: `FOR UPDATE` locks the row until commit. The application must execute the whole block as a single transaction. If the INSERT fails (e.g. FK violation), the ROLLBACK restores the copy status.

Skill 7: Information Retrieval – Full-Text Search and Ranking (Conceptual)

METHOD **Implementing Basic Full-Text Search in SQL**

1. Add a FULLTEXT index on textual columns (MyISAM/InnoDB): `ALTER TABLE Book ADD FULLTEXT(Title, Author);`.
2. Use MATCH … AGAINST: `SELECT *, MATCH(Title,Author) AGAINST('search terms') AS score FROM Book WHERE MATCH(Title,Author) AGAINST('search terms') ORDER BY score DESC;`.
3. Boolean mode: `AGAINST('+database -relational' IN BOOLEAN MODE)` for mandatory/excluded words.
4. Limit results: `LIMIT 20` for pagination.

5. Performance note: Full-text indexes work best on larger texts; for short fields, LIKE with wildcards may suffice.

WORKED EXAMPLE 12 **Full-Text Search for Books**

Statement: Extend the Book table with a FULLTEXT index on Title and Author. Write a query that finds books matching "network security" (both words must appear, order irrelevant) and returns the relevance score, sorted highest first, limited to 10 rows.

SOLUTION

Solution:

```
ALTER TABLE Book ADD FULLTEXT(Title, Author);

SELECT ISBN, Title, Author,
  MATCH(Title, Author) AGAINST('+network +security' IN BOOLEAN MODE) AS relevance
FROM Book
WHERE MATCH(Title, Author) AGAINST('+network +security' IN BOOLEAN MODE)
ORDER BY relevance DESC
LIMIT 10;
```

Explanation: The + operator in Boolean mode requires the word to be present. The relevance score is a floating-point number; higher means more relevant. This technique is examinable for the "Information Retrieval" part of the syllabus.

8.4 Hands-on Project: Building & Binding a Complete Database

Skill 6: DDL for Schema Creation and Conceptual Web-Form Binding

METHOD **From Relational Schema to DDL and Form Binding**

1. Write CREATE TABLE statements: Map each relation: column names, data types (INT, VARCHAR(n), DATE, BOOLEAN), NOT NULL, PRIMARY KEY, FOREIGN KEY REFERENCES.
2. Add indexes: CREATE INDEX on foreign-key columns and frequently searched columns (e.g. Book.Title).
3. Define constraints: CHECK for domain rules (e.g. Status IN ('Available','On

Loan','Lost')), UNIQUE for alternate keys.

4. Create the view(s) from Skill 5 for read-only access.
5. Web-form binding (conceptual):
   - HTML form fields correspond to table columns (e.g. `<input name="RegNo">`).
   - Server-side script (PHP, Python, Node.js) receives POST data, validates, then executes parameterised INSERT/UPDATE (prepared statements) to avoid SQL injection.
   - Dropdown lists for foreign keys: populate from referenced tables (e.g. SELECT ISBN, Title FROM Book for a "Choose Book" dropdown).
   - Feedback: On success, redirect to a confirmation page; on error, re-display form with error messages.
6. Test with sample data: INSERT a few rows, run the view, simulate a form submission.

WORKED EXAMPLE 13 **DDL for Library & Borrow Form Sketch**

Statement: Produce the full DDL (MySQL dialect) for the library schema (Student, Book, Copy, Borrows). Then outline the HTML/PHP code for a "Borrow Book" form that lets a librarian select a student and an available copy, and submits to a script that runs the transaction from Skill 5.

SOLUTION

Solution:

1. DDL:

```
CREATE DATABASE SchoolLibrary;
USE SchoolLibrary;

CREATE TABLE Student (
  RegNo VARCHAR(10) NOT NULL,
  Name VARCHAR(100) NOT NULL,
  Class VARCHAR(20) NOT NULL,
  Email VARCHAR(100),
  PRIMARY KEY (RegNo)
);

CREATE TABLE Book (
  ISBN VARCHAR(13) NOT NULL,
  Title VARCHAR(200) NOT NULL,
  Author VARCHAR(100) NOT NULL,
  Publisher VARCHAR(100),
  Year YEAR,
  PRIMARY KEY (ISBN)
);

CREATE TABLE Copy (
  ISBN VARCHAR(13) NOT NULL,
  CopyID INT NOT NULL,
  Status ENUM('Available','On Loan','Lost') NOT NULL DEFAULT 'Available',
  PRIMARY KEY (ISBN, CopyID),
  FOREIGN KEY (ISBN) REFERENCES Book(ISBN)
    ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE Borrows (
  RegNo VARCHAR(10) NOT NULL,
  ISBN VARCHAR(13) NOT NULL,
  CopyID INT NOT NULL,
  DateBorrowed DATE NOT NULL,
  DueDate DATE NOT NULL,
  Returned BOOLEAN NOT NULL DEFAULT FALSE,
  PRIMARY KEY (RegNo, ISBN, CopyID, DateBorrowed),
  FOREIGN KEY (RegNo) REFERENCES Student(RegNo)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  FOREIGN KEY (ISBN, CopyID) REFERENCES Copy(ISBN, CopyID)
```

2. HTML form (borrow.html):

```
<form action="borrow_process.php" method="post">
  <label>Student:
    <select name="RegNo" required>
      <option value="">-- Choose --</option>
      <?php
      $res = $pdo->query("SELECT RegNo, Name FROM Student ORDER BY Name");
      while($row=$res->fetch()) {
        echo "<option value='{$row['RegNo']}'>{$row['Name']} ({$row['RegNo']})</option>";
      }
      ?>
    </select>
  </label><br><br>
  <label>Available Copy:
    <select name="copy" required>
      <option value="">-- Choose --</option>
      <?php
      $sql = "SELECT c.ISBN, c.CopyID, b.Title
        FROM Copy c JOIN Book b ON c.ISBN=b.ISBN
        WHERE c.Status='Available' ORDER BY b.Title";
      $res = $pdo->query($sql);
      while($row=$res->fetch()) {
        $val = $row['ISBN'].'|'.$row['CopyID'];
        echo "<option value='$val'>{$row['Title']} (Copy {$row['CopyID']})</option>";
      }
      ?>
    </select>
  </label><br><br>
  <button type="submit">Record Loan</button>
</form>
```

3. PHP processing (borrow_process.php) – key excerpt:

```php
<?php
session_start();
require 'db_connect.php'; // $pdo with ERRMODE_EXCEPTION

$RegNo = $_POST['RegNo'];
list($ISBN, $CopyID) = explode('|', $_POST['copy']);
$today = date('Y-m-d');
$due = date('Y-m-d', strtotime('+14 days'));

try {
  $pdo->beginTransaction();
  // lock copy
  $stmt = $pdo->prepare("SELECT Status FROM Copy WHERE ISBN=? AND CopyID=? FOR UPDATE");
  $stmt->execute([$ISBN, $CopyID]);
  $copy = $stmt->fetch();
  if (!$copy || $copy['Status'] !== 'Available') {
    throw new Exception('Copy not available');
  }
  // insert loan
  $stmt = $pdo->prepare(
    "INSERT INTO Borrows (RegNo, ISBN, CopyID, DateBorrowed, DueDate, Returned)"
    . " VALUES (?, ?, ?, ?, ?, FALSE)"
  );
  $stmt->execute([$RegNo, $ISBN, $CopyID, $today, $due]);
  // update copy status
  $stmt = $pdo->prepare("UPDATE Copy SET Status='On Loan' WHERE ISBN=? AND CopyID=?");
  $stmt->execute([$ISBN, $CopyID]);
  $pdo->commit();
  header('Location: success.php');
} catch (Exception $e) {
  $pdo->rollBack();
  echo "Error: " . $e->getMessage();
}
?>
```

Key points: Use prepared statements (placeholders ?) to prevent SQL injection.

The transaction mirrors Skill 5. The form dynamically populates dropdowns from the database, ensuring referential integrity at the UI level.

Skill 8: Integrating All Steps – Mini Project Checklist

METHOD **End-to-End Database Project Workflow**

1. Requirements gathering: Interview stakeholders, write user stories.
2. ER modelling: Draw ER diagram (Skill 1), validate with users.
3. Relational mapping & normalisation: Apply Skills 2 & 3, produce schema diagram.
4. DDL implementation: Write CREATE statements (Skill 6), run on DBMS.
5. Sample data loading: INSERT realistic test data (at least 5 rows per table).
6. Core queries: Implement the 5–10 most frequent queries (Skill 4).
7. Advanced features: Views, transactions, full-text search (Skills 5 & 7).
8. Application binding: Build a simple CRUD web interface (Skill 6).
9. Testing: Unit test each query, test transaction rollback, test form validation.
10. Documentation: Data dictionary, ER diagram, schema diagram, user manual.
11. Exam readiness: Be able to reproduce any step on paper within 30 minutes.

WORKED EXAMPLE 14 **Project Checklist Applied to Library System**

Statement: Summarise how you would execute the mini-project checklist for the school library system, highlighting the deliverables at each stage and the GCE-style evidence you would present.

SOLUTION

Solution:

1. Requirements: Document: “Librarian needs to register students, catalogue books, record loans, generate overdue list.” User stories: “As a librarian I want to issue a book so that the student can take it home.”
2. ER Diagram: Deliverable: Hand-drawn or software (e.g. Draw.io) diagram with entities, attributes, relationships, cardinalities. Evidence: Photograph/scan in report.

3. Relational Schema & Normalisation: Deliverable: Table definitions with PK/FK, normalisation proof (FD list, decomposition steps). Evidence: Written steps from Skill 3.
4. DDL Script: File `library_schema.sql` containing all CREATE statements (Skill 6). Evidence: Printout of script and successful execution screenshot.
5. Test Data: `INSERT` scripts for 10 students, 15 books, 3 copies each, 20 loans. Evidence: SELECT * FROM each table showing rows.
6. Core Queries: Five queries: (a) List all books by author, (b) Current loans per student, (c) Overdue report (Skill 4), (d) Most borrowed books, (e) Students with no loans. Evidence: SQL code + result screenshots.
7. Advanced: View `OverdueLoans` (Skill 5), transaction script for borrowing, full-text index + search query (Skill 7). Evidence: View definition, transaction pseudocode, search results.
8. Web Interface: Three pages: `borrow.html`, `return.html`, `overdue.php`. Use PHP/PDO with prepared statements. Evidence: Screenshots of forms, code snippets.
9. Testing: Test cases: (i) Borrow available copy → success, (ii) Borrow same copy twice → rollback, (iii) Search "network" → returns relevant books. Evidence: Test log table.
10. Documentation: Data dictionary (table, column, type, constraints), ER diagram, schema diagram, user guide (2 pages). Evidence: Compiled PDF.
11. Exam Preparation: Practice drawing the ER diagram from memory, writing the normalisation steps, and coding the overdue query under timed conditions.

Final tip: In the GCE practical paper, you may be given a scenario and asked to produce the ER diagram, the normalised tables, and two SQL queries. Mastering this checklist guarantees you cover every required artefact.

DID YOU KNOW? **Database Systems in Cameroon**

In Cameroon, many secondary schools and universities use MySQL or PostgreSQL for managing student records, library catalogues, and examination results. The GCE Advanced Level Computer Science practical paper often requires candidates to design a database for a local context (e.g., a school library, a clinic, or a small business). Understanding how to move from a narrative description to a normalised schema and then to a working SQL implementation is a core competency examined both in theory and practical papers.

KEY POINTS **Key Points for Revision**

- ER diagrams: entities (strong/weak), attributes (simple/composite/multi-valued/derived), relationships with cardinalities and participation.
- Mapping rules: strong entity → table; weak entity → table with composite PK; 1:N → FK on N side; M:N → junction table.
- Normalisation: 1NF (atomic), 2NF (no partial FD), 3NF (no transitive FD), BCNF (every determinant is a superkey).
- SQL: SELECT with JOIN, GROUP BY, HAVING; subqueries (correlated, derived tables); views; transactions (START TRANSACTION, COMMIT, ROLLBACK).
- DDL: CREATE TABLE with PK, FK, CHECK, ENUM, indexes; use prepared statements in application code to prevent SQL injection.
- Full-text search: FULLTEXT index, MATCH … AGAINST in Boolean mode for relevance ranking.
- Project workflow: requirements → ER → schema → DDL → data → queries → views/transactions → web forms → test → document.

WARNING **Common Examination Pitfalls**

- Forgetting to underline primary keys in ER diagrams.
- Misidentifying weak entities (e.g., treating Copy as strong).
- Missing functional dependencies during normalisation, leading to incomplete decomposition.
- Using WHERE instead of HAVING for aggregate conditions.
- Writing SQL vulnerable to injection (concatenating strings instead of prepared statements).
- Omitting transaction control for multi-step updates (e.g., borrowing a book).
- Not escaping special characters in LaTeX code listings (

EXERCISE 6 **Practice Questions**

- Draw the ER diagram for a university course registration system: Students enrol in Courses; each Course is taught by one Lecturer; a Lecturer may teach many Courses. A Student receives a Grade for each Course. Include attributes: Student(ID, Name, Email), Course(Code, Title, Credits), Lecturer(ID, Name, Department), Grade. Indi-

cate cardinalities and participation.

- Given the universal relation R(StudentID, StudentName, CourseCode, CourseTitle, LecturerID, LecturerName, Grade) with FDs: StudentID → StudentName; CourseCode → CourseTitle, LecturerID; LecturerID → LecturerName; (StudentID, CourseCode) → Grade. Normalise to BCNF, showing each step.
- Write a SQL query to list all lecturers who teach more than 3 courses, showing LecturerID, LecturerName, and the number of courses.
- Create a view `StudentCourseView` that shows StudentName, CourseTitle, LecturerName, Grade for all enrolments.
- Write a transaction (in SQL pseudocode) to enrol a student in a course: check that the student is not already enrolled, insert a row in Enrolment with Grade = NULL, and increment a `EnrolledCount` column in the Course table. Roll back if any step fails.

SOLUTION **Exercise 1**

ER Diagram:

- Entities: `Student` (PK: StudentID), `Course` (PK: CourseCode), `Lecturer` (PK: LecturerID).
- Relationships: `Enrols` between Student and Course (M:N, attributes: Grade). `Teaches` between Lecturer and Course (1:N, Lecturer 1 – N Course). Participation: Student partial in Enrols, Course total in Enrols (a course exists even if no students? Usually partial both, but syllabus may expect total for Course? We'll state assumptions). Lecturer total in Teaches (a lecturer must teach at least one course? Usually partial). Course total in Teaches (a course must have a lecturer).
- Attributes: as given. Grade on Enrols.

SOLUTION **Exercise 2**

Normalisation to BCNF:

1. FDs: as given. Candidate key: (StudentID, CourseCode).
2. 1NF: atomic.
3. 2NF: Partial FDs: StudentID → StudentName; CourseCode → CourseTitle, LecturerID. Decompose:
   - Student(StudentID, StudentName)

- Course(CourseCode, CourseTitle, LecturerID)
- Lecturer(LecturerID, LecturerName)
- Enrolment(StudentID, CourseCode, Grade)

4. 3NF: In Course, LecturerID → LecturerName (transitive). Already separated. All 3NF.
5. BCNF: Check each relation: Student (StudentID PK), Lecturer (LecturerID PK), Course (CourseCode PK; LecturerID is FK, not a determinant of non-key), Enrolment ((StudentID, CourseCode) PK). All BCNF.

SOLUTION **Exercise 3**

SQL Query:

```
SELECT l.LecturerID, l.LecturerName, COUNT(c.CourseCode) AS NumCourses
FROM Lecturer l
JOIN Course c ON l.LecturerID = c.LecturerID
GROUP BY l.LecturerID, l.LecturerName
HAVING COUNT(c.CourseCode) > 3;
```

SOLUTION **Exercise 4**

View:

```
CREATE VIEW StudentCourseView AS
SELECT s.StudentName, c.CourseTitle, l.LecturerName, e.Grade
FROM Student s
JOIN Enrolment e ON s.StudentID = e.StudentID
JOIN Course c ON e.CourseCode = c.CourseCode
JOIN Lecturer l ON c.LecturerID = l.LecturerID;
```

SOLUTION **Exercise 5**

Transaction Pseudocode:

```
START TRANSACTION;
-- Check existing enrolment
SELECT COUNT(*) FROM Enrolment WHERE StudentID = ? AND CourseCode = ?;
IF count > 0 THEN ROLLBACK; SIGNAL 'Already enrolled'; END IF;
-- Insert enrolment
INSERT INTO Enrolment (StudentID, CourseCode, Grade) VALUES (?, ?, NULL);
-- Increment course count
UPDATE Course SET EnrolledCount = EnrolledCount + 1 WHERE CourseCode = ?;
COMMIT;
```

9 EXERCISES

9.1 I check my knowledge

EXERCISE 7 **Multiple choice questions**

For each question, choose the single correct answer.

1. A relation is in First Normal Form (1NF) when:

   (a) it contains no transitive dependency;

   (b) every attribute holds only atomic (indivisible) values;

   (c) it has exactly one foreign key;

   (d) it contains no duplicate rows at all.

2. In the relation `ENROL(StudID, SubjectCode, StudName, SubjectName)` with key `(StudID, SubjectCode)`, the attribute `StudName` depends only on `StudID`. This is an example of:

   (a) a transitive dependency;

   (b) a partial dependency;

   (c) a full functional dependency;

   (d) a multivalued dependency.

3. The SQL keyword used to eliminate duplicate rows from the result of a `SELECT` is:

   (a) `UNIQUE`;

   (b) `ONLY`;

   (c) `DISTINCT`;

   (d) `DIFFERENT`.

4. In an ER diagram, a many-to-many relationship between two entities is implemented in the relational schema by:

   (a) a foreign key placed in either of the two tables;

   (b) a new junction (associative) table;

   (c) merging the two entities into one;

   (d) duplicating the primary key of one entity.

EXERCISE 8 **True or false — justify**

State whether each assertion is true or false, and justify your answer in one or two sentences.

1. A table that is in 2NF is automatically in 3NF.
2. Every table in a relational database must have a primary key.
3. The SQL statement `DELETE FROM PAYMENT;` removes all the rows of the table `PAYMENT` but keeps the table structure.
4. A foreign key in a child table can contain a value that does not exist in the parent table.

EXERCISE 9 **Key definitions**

Define each of the following terms in one or two sentences, giving an example where appropriate:

1. **Primary key**;
2. **Foreign key**;
3. **Functional dependency**;
4. **Transitive dependency**;
5. **Cardinality** (in an ER diagram).

EXERCISE 10 **Normal forms — direct check**

Consider the relation `SALE(SaleID, Date, CustID, CustName, CustTown, TownRegion)` with primary key `SaleID`, where `CustID` determines `CustName` and `CustTown`, and `CustTown` determines `TownRegion`.

1. State the highest normal form satisfied by this relation. Justify.
2. Identify the transitive dependency present.
3. Decompose the relation into relations in 3NF, underlining each primary key.

9.2 I practise

EXERCISE 11 Normalising a GCE registration table

The West Regional Delegation stores candidate registrations in a single flat table:

CandNo	CandName	CentreNo	CentreName	Subjects
C001	Acha Marie	101	GBHS Bamenda	Maths, Physics
C002	Ndi Paul	101	GBHS Bamenda	Chemistry
C003	Eko Ruth	102	GBHS Limbe	Maths, Biology

1. Give two reasons why this table is not in 1NF.
2. Identify one insertion anomaly, one update anomaly and one deletion anomaly that could occur.
3. Decompose the table into a set of 3NF relations. Underline the primary keys and indicate the foreign keys.

EXERCISE 12 ER diagram of a Douala pharmacy

A pharmacy in Douala wants a database with the following requirements: a medicine has a code, a name and a unit price; a supplier has a number, a name and a town; a sale has a number and a date. A supplier supplies several medicines and a medicine can be supplied by several suppliers. A sale concerns one or more medicines, with a quantity for each; every medicine may appear in many sales.

1. Draw the complete ER diagram: entities, attributes (underline identifiers), relationships and cardinalities.
2. Derive the corresponding relational schema, underlining primary keys and marking foreign keys with an asterisk (*).

EXERCISE 13 CREATE TABLE and INSERT

Write the SQL statements for a school fees database:

1. `CREATE TABLE STUDENT` with attributes `StudID` (integer, primary key), `Name` (text up to 40 characters, not null), `Gender` (single character).
2. `CREATE TABLE PAYMENT` with attributes `PayID` (integer, primary key), `StudID` (integer, foreign key referencing `STUDENT`), `Amount` (integer, not null), `PayDate` (date).
3. Three `INSERT` statements adding one student and two payments for that student.

EXERCISE 14 **SELECT queries with JOIN, GROUP BY and HAVING**

Given the tables `CLASS(ClassID, ClassName)` and `STUDENT(StudID, Name, ClassID)`, write SQL queries to:

1. display the names of all students of the class `'Upper Sixth A'`;
2. display the number of students per class (class name and count);
3. display only the classes having more than 50 students;
4. display the names of students together with their class name, sorted alphabetically by student name.

9.3 I prepare for the GCE

EXERCISE 15 **Structured question — normalisation (20 marks)**

A cybercafé in Yaoundé keeps its billing records in one table:
`BILL(BillNo, Date, ClientID, ClientName, MachineNo, MachineType, Hours, RatePerHour)`
with primary key `BillNo`. A client is identified by `ClientID`; a machine is identified by `MachineNo` and `MachineNo` determines `MachineType`; each machine type has a fixed `RatePerHour`.

1. Define the term functional dependency and list all the functional dependencies in this table. (4 marks)
2. State the highest normal form of `BILL`, justifying your answer. (3 marks)
3. Explain, with an example from this table, one update anomaly and one deletion anomaly. (4 marks)
4. Normalise the table step by step into 3NF, showing the relations obtained at each stage. Underline primary keys and mark foreign keys. (9 marks)

EXERCISE 16 **Paper-tracing of SQL queries (15 marks)**

Consider the following database content:
`CLASS`

ClassID	ClassName
1	Lower Sixth Sc
2	Upper Sixth A
3	Upper Sixth C

`STUDENT`

StudID	Name	ClassID	FeesPaid
10	Bih Anne	2	50000
11	Tabi John	2	35000
12	Meka Solange	3	50000
13	Fon Kevin	1	25000
14	Ngo Carine	2	50000

Predict the exact output (columns and rows, in the correct order) of each query:

1. `SELECT Name, FeesPaid FROM STUDENT WHERE FeesPaid >= 35000 ORDER BY FeesPaid DESC;` (3 marks)
2. `SELECT ClassName, COUNT(*) AS Nb FROM CLASS C JOIN STUDENT S ON C.ClassID = S.ClassID GROUP BY ClassName;` (5 marks)
3. `SELECT ClassName, SUM(FeesPaid) AS Total FROM CLASS C JOIN STUDENT S ON C.ClassID = S.ClassID GROUP BY ClassName HAVING SUM(FeesPaid) > 60000;` (5 marks)
4. State the purpose of the `HAVING` clause and explain how it differs from `WHERE`. (2 marks)

EXERCISE 17 **Many-to-many relationships and safe updates (15 marks)**

In a school database, a student registers for several subjects and each subject is followed by several students.

1. Explain clearly why the many-to-many relationship between `STUDENT` and `SUBJECT` cannot be represented by a simple foreign key in either table. (4 marks)
2. Draw the ER diagram of this relationship with its cardinalities. (3 marks)
3. Write the `CREATE TABLE` statement of the junction table `REGISTER(StudID, SubjectCode, Term)`, choosing appropriate data types, the composite primary key and the two foreign keys. (5 marks)
4. The bursar wanted to update the fees of the class `'Upper Sixth A'` only, and typed: `UPDATE STUDENT SET FeesPaid = 50000;`

All the rows of the table were modified. Identify the missing clause and rewrite the correct statement. State one precaution to take before running any `UPDATE` or `DELETE` statement. (3 marks)

10 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple choice questions**

1. Correct answer: b). First Normal Form (1NF) requires that every attribute of every tuple contains a single **atomic** (indivisible) value — no lists, no repeating groups, no nested relations. Option (a) describes the elimination of transitive dependencies, which is the goal of 3NF. Option (d) — uniqueness of rows — is a fundamental property of any relation (guaranteed by the primary key), not specific to 1NF. Option (c) is unrelated to normalisation.

2. Correct answer: b). `StudName` depends on `StudID`, which is only part of the composite primary key `(StudID, SubjectCode)`. This is precisely the definition of a **partial dependency**, the anomaly that Second Normal Form (2NF) eliminates. A relation is in 2NF iff it is in 1NF and no non-key attribute depends on a proper subset of any candidate key.

3. Correct answer: c). `SELECT DISTINCT` eliminates duplicate rows from the result set. `UNIQUE` exists in SQL but only as a table constraint (e.g., `UNIQUE (Email)`) or in `CREATE INDEX`, never as a keyword in the `SELECT` list.

4. Correct answer: b). A many-to-many relationship cannot be represented by a single foreign key in either table. It is implemented by a **junction table** (also called an associative or linking table) whose primary key is the combination of the two entity identifiers, each stored as a foreign key referencing the respective parent table.

SOLUTION **Exercise 2 — True or false — justify**

1. False. Second Normal Form (2NF) only eliminates partial dependencies (a non-key attribute depending on part of a composite candidate key). It says nothing about transitive dependencies (a non-key attribute determining another non-key attribute). A table can therefore be in 2NF but not in 3NF. Example: `R(A,B,C)` with key `A` and dependencies `A → B`, `B → C` is in 2NF (key is simple) but not in 3NF because of the transitive dependency `A → C` via `B`.

2. True. Every relation (table) in a relational database must have a primary key — a minimal set of attributes that uniquely identifies each tuple. Without a primary key, duplicate rows could not be distinguished, and referential integrity (foreign keys referencing this table) would be impossible to enforce.

3. True. `DELETE FROM PAYMENT;` (without a `WHERE` clause) removes all rows from the table but leaves the table structure — columns, data types, constraints, indexes — intact in the database catalogue. It is `DROP TABLE PAYMENT;` that destroys the table structure

itself.

4. False. This would violate **referential integrity**. The rule is: every non-`NULL` foreign key value in the child (referencing) table must match an existing primary key value in the parent (referenced) table. If the parent row does not exist, the foreign key constraint rejects the insertion or update (unless the foreign key column allows `NULL` and the value is `NULL`).

SOLUTION **Exercise 3 — Key definitions**

1. Primary key: an attribute (or minimal set of attributes) whose value uniquely identifies each row of a table; it can never be `NULL` nor duplicated. Example: `StudID` in `STUDENT`.
2. Foreign key: an attribute (or set of attributes) of a (child) table that references the primary key of another (parent) table, creating a link between them and enforcing referential integrity. Example: `ClassID` in `STUDENT` referencing `CLASS(ClassID)`.
3. Functional dependency: a constraint $X \rightarrow Y$ (where X and Y are sets of attributes) meaning that whenever two rows agree on the value of X, they must agree on the value of Y — i.e., X functionally determines Y. Example: `ClientID` → `ClientName`.
4. Transitive dependency: an indirect functional dependency $X \rightarrow Y$ and $Y \rightarrow Z$ where Y is not a candidate key (and Z is not a subset of Y), so that X determines Z through Y. This is the anomaly removed by 3NF. Example: `SaleID` → `CustTown` → `TownRegion`.
5. Cardinality: in an ER diagram, the pair of minimum and maximum numbers of times an entity occurrence participates in a relationship, written (min, max). For example, $(1, n)$ means "at least once, possibly many times" (mandatory participation with no upper bound); $(0, 1)$ means "at most once" (optional participation).

SOLUTION **Exercise 4 — Normal forms — direct check**

1. The relation is in 2NF (highest normal form satisfied). Justification:
   - 1NF: all attribute values are atomic (no repeating groups, no composite attributes).
   - 2NF: the primary key `SaleID` is a single attribute; therefore no non-key attribute can depend on a proper subset of the key — partial dependencies are impossible. The relation satisfies 2NF.
   - Not 3NF: there exists a transitive dependency `SaleID` → `CustTown` → `TownRegion`. The non-key attribute `TownRegion` is determined by another non-

key attribute `CustTown`, violating 3NF.

2. The transitive dependency chain is: `SaleID` → `CustID` → `CustTown` → `TownRegion`. In the given relation, `CustTown` and `TownRegion` are both non-key attributes, and `TownRegion` depends on the key `SaleID` only through `CustTown`.
3. 3NF decomposition (primary keys underlined, foreign keys marked with *):
   - `SALE(SaleID, Date, CustID*)`
   - `CUSTOMER(CustID, CustName, CustTown*)`
   - `TOWN(CustTown, TownRegion)`

   Each non-key attribute now depends on the key, the whole key, and nothing but the key (Codd's informal definition of 3NF). The foreign key `CustTown*` in `CUSTOMER` references `TOWN(CustTown)`.

SOLUTION **Exercise 5 — Normalising a GCE registration table**

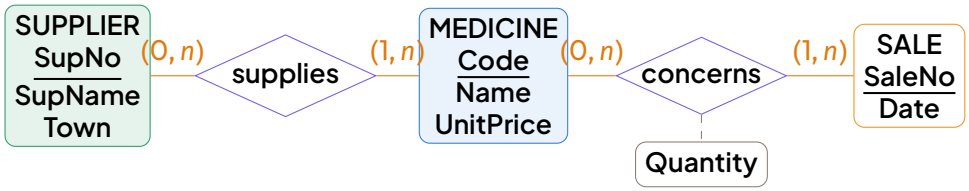
1. Two reasons why the table is not in 1NF:
   - The attribute `Subjects` is not atomic: it contains a list of several values (e.g., "Maths, Physics") in a single cell.
   - Consequently there is a repeating group: the number of subjects varies from one candidate to another, so a single cell cannot hold them all atomically without violating the relational model's requirement that each attribute hold exactly one value per tuple.
2. Anomalies caused by the unnormalised structure:
   - Insertion anomaly: a new centre (e.g., GBHS Buea, centre 103) cannot be recorded until at least one candidate registers there, because centre data (`CentreNo`, `CentreName`) only exist inside candidate rows.
   - Update anomaly: if GBHS Bamenda changes its name, the name must be corrected in every row where `CentreNo = 101`; missing one row creates inconsistency (two different names for the same centre number).
   - Deletion anomaly: deleting candidate C002 (the only remaining candidate of centre 101) would also delete the information about that centre (`CentreNo`, `CentreName`), even though the centre still exists.
3. 3NF decomposition (primary keys underlined, foreign keys marked *):
   - `CENTRE(CentreNo, CentreName)`
   - `CANDIDATE(CandNo, CandName, CentreNo*)`

- SUBJECT(SubjectCode, SubjectName) (reference table for subjects)
- REGISTRATION(CandNo*, SubjectCode*) (junction table resolving the many-to-many relationship between candidates and subjects)

The many-to-many relationship "a candidate takes several subjects, a subject is taken by several candidates" is correctly resolved by REGISTRATION; centre data are stored once in CENTRE. All three anomalies disappear.

SOLUTION **Exercise 6 — ER diagram of a Douala pharmacy**

1. ER diagram: three entities — MEDICINE (Code, Name, UnitPrice), SUPPLIER (SupNo, SupName, Town), SALE (SaleNo, Date) — and two many-to-many relationships:
   - supplies between SUPPLIER (0, *n*) and MEDICINE (1, *n*): a medicine must have at least one supplier (mandatory), a supplier may supply none yet (optional).
   - concerns between SALE (1, *n*) and MEDICINE (0, *n*), carrying the attribute Quantity (the quantity of that medicine in that sale).



ER diagram of the pharmacy database (identifiers underlined).

2. Relational schema derived from the ER diagram (primary keys underlined, foreign keys marked *):
   - MEDICINE(Code, Name, UnitPrice)
   - SUPPLIER(SupNo, SupName, Town)
   - SALE(SaleNo, Date)
   - SUPPLIES(SupNo*, Code*)
   - CONCERNS(SaleNo*, Code*, Quantity)

Each many-to-many relationship becomes a junction table whose composite primary key is the pair of the two identifiers. Quantity — an attribute of the concerns relationship — is stored in CONCERNS.

SOLUTION Exercise 7 — CREATE TABLE and INSERT

1. Creation of the parent table `STUDENT`:

```
CREATE TABLE STUDENT (
  StudID INTEGER PRIMARY KEY,
  Name VARCHAR(40) NOT NULL,
  Gender CHAR(1)
);
```

2. Creation of the child table `PAYMENT` with its foreign key referencing `STUDENT`:

```
CREATE TABLE PAYMENT (
  PayID INTEGER PRIMARY KEY,
  StudID INTEGER,
  Amount INTEGER NOT NULL,
  PayDate DATE,
  FOREIGN KEY (StudID) REFERENCES STUDENT(StudID)
);
```

3. Insertion of one student and two payments (order matters!):

```
INSERT INTO STUDENT VALUES (1, 'Acha Marie', 'F');
INSERT INTO PAYMENT VALUES (101, 1, 25000, '2025-01-15');
INSERT INTO PAYMENT VALUES (102, 1, 25000, '2025-03-10');
```

Remark: the student must be inserted before the payments, otherwise the foreign key constraint on `StudID` would be violated (referential integrity requires the referenced parent row to exist).

SOLUTION Exercise 8 — SELECT queries with JOIN, GROUP BY and HAVING

Assume tables `STUDENT(StudID, Name, ClassID, FeesPaid)` and `CLASS(ClassID, ClassName)`.

1. Students of ‘Upper Sixth A’:

```
SELECT S.Name
FROM STUDENT S JOIN CLASS C ON S.ClassID = C.ClassID
WHERE C.ClassName = 'Upper Sixth A';
```

2. Number of students per class:
   - `SELECT C.ClassName, COUNT(*) AS NbStudents`
   - `FROM CLASS C JOIN STUDENT S ON C.ClassID = S.ClassID`
   - `GROUP BY C.ClassName;`
3. Classes with more than 50 students:
   - `SELECT C.ClassName, COUNT(*) AS NbStudents`
   - `FROM CLASS C JOIN STUDENT S ON C.ClassID = S.ClassID`
   - `GROUP BY C.ClassName`
   - `HAVING COUNT(*) > 50;`

   The condition on the aggregate `COUNT(*)` must be in `HAVING`, not in `WHERE` (which filters rows before grouping and cannot contain aggregates).
4. Students with their class, sorted by name ascending:
   - `SELECT S.Name, C.ClassName`
   - `FROM STUDENT S JOIN CLASS C ON S.ClassID = C.ClassID`
   - `ORDER BY S.Name ASC;`

SOLUTION **Exercise 9 — Structured question — normalisation (20 marks)**

1. (4 marks) Definition (1 mark): a functional dependency $X \to Y$ means that the value of X uniquely determines the value of Y: two rows with the same X must have the same Y. Dependencies in the BILL table (3 marks, approx. 1 mark per correct group):
   - `BillNo` → `Date, ClientID, MachineNo, Hours` (the key determines all other attributes);
   - `ClientID` → `ClientName;`
   - `MachineNo` → `MachineType;`
   - `MachineType` → `RatePerHour.`
2. (3 marks) Highest normal form: 2NF (1 mark). Justification (2 marks): all values are atomic (1NF); the key `BillNo` is a single attribute so no partial dependency on part of a composite key can exist (2NF); but there are transitive dependencies, e.g., `BillNo` → `MachineNo` → `MachineType` → `RatePerHour`, so the table is not in 3NF.
3. (4 marks) Anomalies in the unnormalised table:
   - Update anomaly (2 marks): if the rate of a machine type (say “Desktop”) changes,

`RatePerHour` must be updated in every bill row involving that type; forgetting one row leaves two different rates for the same type — an inconsistency.

- Deletion anomaly (2 marks): if the only bill ever recorded for client C017 is deleted, all information about that client (`ClientID`, `ClientName`) disappears from the database, although the client still exists.

4. (9 marks) Step-by-step normalisation (examiner expects explicit stages):
   - 1NF (2 marks): data already atomic — `BILL(BillNo, Date, ClientID, ClientName, MachineNo, MachineType, Hours, RatePerHour)`.
   - 2NF (2 marks): the key is simple (`BillNo`), so no partial dependency exists; the relation is already in 2NF (state this explicitly).
   - 3NF (5 marks): remove the transitive dependencies by extracting one table per determinant:
     - `BILL(BillNo, Date, ClientID*, MachineNo*, Hours)`
     - `CLIENT(ClientID, ClientName)`
     - `MACHINE(MachineNo, MachineType*)`
     - `MACHINETYPE(MachineType, RatePerHour)`

   Examiner's expectations: the candidate must name each normal form reached at each stage, underline every primary key, mark foreign keys with *, and justify each step by the dependency removed. A frequent error costing marks is jumping directly to the final schema without showing the intermediate stages.

SOLUTION **Exercise 10 — Paper-tracing of SQL queries (15 marks)**

Given the sample data:

Name	ClassName	FeesPaid
Bih Anne	Upper Sixth A	50000
Fon Kevin	Lower Sixth Sc	25000
Meka Solange	Upper Sixth A	50000
Ngo Carine	Upper Sixth A	50000
Tabi John	Upper Sixth A	35000

1. (3 marks) Query: `SELECT Name, FeesPaid FROM STUDENT WHERE FeesPaid >= 35000 ORDER BY FeesPaid DESC;` Rows kept: `FeesPaid >= 35000` eliminates Fon Kevin (25000); sorting descending by `FeesPaid` (ties keep original table order):

Name	FeesPaid
Bih Anne	50000
Meka Solange	50000
Ngo Carine	50000
Tabi John	35000

2. (5 marks) Query: `SELECT ClassName, COUNT(*) AS Nb FROM STUDENT JOIN CLASS ON STUDENT.ClassID = CLASS.ClassID GROUP BY ClassName;` The join links each student to his/her class; `GROUP BY ClassName` counts per class:

ClassName	Nb
Lower Sixth Sc	1
Upper Sixth A	3
Upper Sixth C	1

(Upper Sixth A: Bih Anne, Tabi John, Ngo Carine.)

3. (5 marks) Query: `SELECT ClassName, SUM(FeesPaid) AS Total FROM STUDENT JOIN CLASS ON STUDENT.ClassID = CLASS.ClassID GROUP BY ClassName HAVING SUM(FeesPaid) > 60000;` Totals per class: Lower Sixth Sc = 25000; Upper Sixth A = 50000 + 35000 + 50000 = 135000; Upper Sixth C = 50000. `HAVING SUM(FeesPaid) > 60000` keeps only Upper Sixth A:

ClassName	Total
Upper Sixth A	135000

4. (2 marks) `HAVING` filters groups produced by `GROUP BY`, using aggregate functions (e.g., `SUM`, `COUNT`, `AVG`); `WHERE` filters individual rows before grouping and cannot contain aggregate functions.

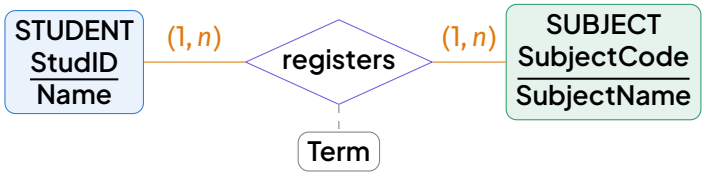
Examiner's expectations: exact column headings (including aliases `Nb`, `Total`), correct row order, and no extra rows. Showing the intermediate computation (sums per class) earns method marks even if a slip occurs.

SOLUTION **Exercise 11 — Many-to-many relationships and safe updates (15 marks)**

1. (4 marks) A foreign key is a single attribute (or set of attributes) holding one value per row, so it can link a row to only one row of the referenced table. Placing `SubjectCode` in `STUDENT` would allow each student to register for only one subject; placing `StudID` in `SUBJECT` would allow each subject to have only one student. Storing lists of values

(e.g., “MATH,PHYS”) would violate 1NF, and repeating rows (one per subject) would duplicate student data massively. Hence a simple foreign key in either table cannot express many-to-many: a separate **junction table** is required.

2. (3 marks) ER diagram for the many-to-many relationship registers between `STUDENT` and `SUBJECT`, with attribute `Term` on the relationship:



Many-to-many relationship between STUDENT and SUBJECT.

3. (5 marks) SQL creation of the junction table `REGISTER`:

```
CREATE TABLE REGISTER (
  StudID INTEGER NOT NULL,
  SubjectCode VARCHAR(10) NOT NULL,
  Term VARCHAR(15),
  PRIMARY KEY (StudID, SubjectCode),
  FOREIGN KEY (StudID) REFERENCES STUDENT(StudID),
  FOREIGN KEY (SubjectCode) REFERENCES SUBJECT(SubjectCode)
);
```

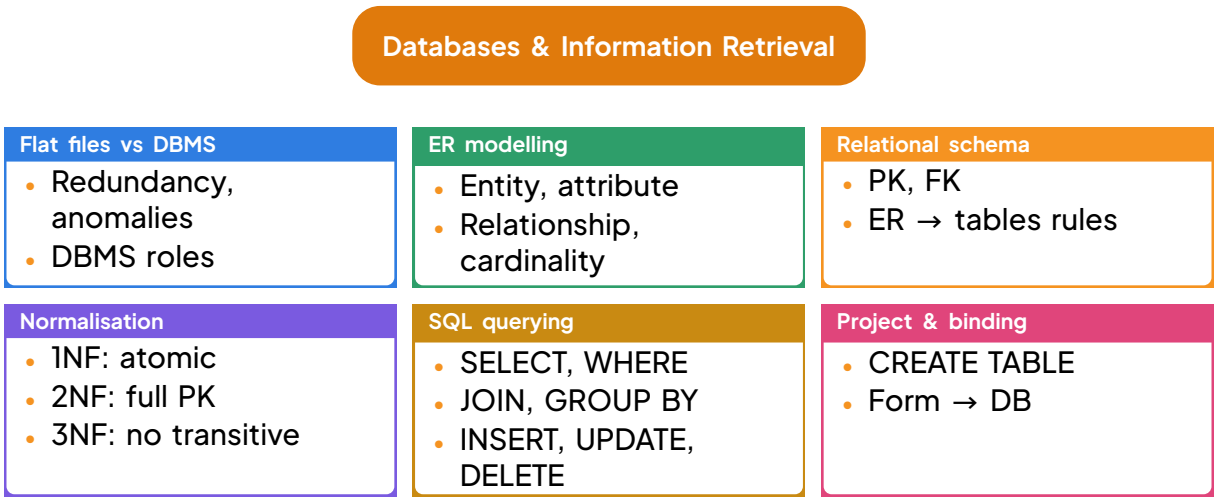
Marks: correct data types (1), composite primary key (2), the two foreign keys (2).

4. (3 marks) The missing clause is `WHERE`: without it, `UPDATE` applies to all rows in the table. Correct statement (using a subquery to find the `ClassID`):

```
UPDATE STUDENT SET FeesPaid = 50000
WHERE ClassID = (SELECT ClassID FROM CLASS WHERE ClassName = 'Upper Sixth A');
```

Precaution (1 mark): before running any `UPDATE` or `DELETE`, first run the corresponding `SELECT` with the same `WHERE` clause to check exactly which rows will be affected (and/or take a backup of the table).

11 SUMMARY SHEET



Mind map of the chapter: from flat files to a complete, normalised and queryable database.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- A **database** is an organised collection of related data, stored so that it can be accessed, managed and updated efficiently; a **DBMS** (Database Management System) is the software that creates, manages, queries and protects it (e.g. MySQL, SQLite, PostgreSQL, Microsoft Access).
- **Flat file**: data stored in a single table or file; it causes **redundancy** (the same data repeated), **inconsistency** (copies that disagree), and insertion, deletion and update **anomalies**.
- **Entity**: an object of the real world about which we store data (e.g. STUDENT); **attribute**: a property of an entity (e.g. name, date of birth); **relationship**: an association between entities, described by its **cardinalities** (1:1, 1:N, M:N).
- **Primary key** (PK): a minimal set of attributes that uniquely identifies each row of a table; it can never be NULL. A **foreign key** (FK) is an attribute (or set of attributes) that references the PK of another table and links the two tables.
- **Functional dependency** $A \rightarrow B$: the value of A determines the value of B (two rows with the same A always have the same B).
- **Referential integrity**: every FK value must either match an existing PK value in the referenced table or be NULL.

Rules and normal forms to know

- ER → relational schema: each entity becomes a table; a 1:N relationship becomes a FK placed on the N side; an M:N relationship becomes a new table whose PK is the combination of the two entity PKs.
- **1NF** (First Normal Form): all attribute values are atomic; no repeating groups and no multi-valued columns.
- **2NF** (Second Normal Form): the table is in 1NF and every non-key attribute depends on the whole primary key (no partial dependency on part of a composite key).
- **3NF** (Third Normal Form): the table is in 2NF and there is no transitive dependency (a non-key attribute must not depend on another non-key attribute).
- Normalisation removes redundancy and therefore removes insertion, update and deletion anomalies.

SQL essentials

- Query: `SELECT col1, col2 FROM table WHERE condition ORDER BY col;`
- Join: `SELECT ... FROM A JOIN B ON A.fk = B.pk;`
- Aggregation: `SELECT dept, COUNT(*) FROM Emp GROUP BY dept HAVING COUNT(*) > 5;`
- Updates: `INSERT INTO t VALUES (...);` `UPDATE t SET c = v WHERE ...;` `DELETE FROM t WHERE ...;`
- Creation: `CREATE TABLE Student (id INTEGER PRIMARY KEY, name VARCHAR(50), class_id INTEGER, FOREIGN KEY (class_id) REFERENCES Class(id));`
- Form binding: a web or client form collects values → the values are validated → they are sent to the DBMS as parameterised INSERT or SELECT statements, and results are displayed back to the user.

Methods

1. Modelling: list the entities → list their attributes and choose each PK → identify the relationships with their cardinalities → draw the ER diagram → convert to tables (FK on the N side; new table for M:N) → check 1NF, then 2NF, then 3NF.
2. Querying: identify the tables needed → choose the join keys → filter rows with WHERE → group and aggregate if required (GROUP BY, HAVING) → sort with ORDER BY.
3. Normalising: at each step, name the dependency that violates the normal form, then decompose the table so that the dependency is preserved in a separate table.

Common mistakes

- Confusing WHERE (filters rows before grouping) with HAVING (filters groups after GROUP BY).

- Forgetting the join condition, which produces a Cartesian product (every row of A paired with every row of B).
- Storing comma-separated lists in one column: this violates 1NF.
- Running UPDATE or DELETE with no WHERE clause: all rows are modified or removed.
- Choosing a non-unique or changeable attribute (e.g. a name) as primary key.
- Placing the FK on the wrong side of a 1:N relationship (it must be on the N side).

GCE tips

- In ER questions, always state the cardinalities on the diagram and justify the FK placement in one sentence.
- Show normalisation step by step (1NF → 2NF → 3NF), naming the dependency removed at each step; examiners award marks for the justification, not only the final tables.
- Read SQL questions twice: underline the output columns, the tables, and the conditions before writing anything.
- Practise writing one SELECT combining JOIN + GROUP BY + HAVING: it is the classic exam question.
- Remember that every SQL statement ends with a semicolon, and text values are written between single quotes.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+